

Swarm: A Distributed Ledger-based Framework to Enhance Air Traffic Control Security Using ADS-B Protocol

GABRIELE DIGREGORIO, Dipartimento di Elettronica, Informazione e Bioingegneria, Politecnico di Milano, Milano, Italy

EDOARDO SAPUTELLI, Dipartimento di Elettronica, Informazione e Bioingegneria, Politecnico di Milano, Milano, Italy

STEFANO LONGARI, Dipartimento di Elettronica, Informazione e Bioingegneria, Politecnico di Milano, Milano, Italy

MICHELE CARMINATI, Dipartimento di Elettronica, Informazione e Bioingegneria, Politecnico di Milano, Milan, Italy

STEFANO ZANERO, Dipartimento di Elettronica, Informazione e Bioingegneria, Politecnico di Milano, Milan, Italy

In aviation, safety is paramount, with air traffic control (ATC) playing a crucial role in monitoring aircraft to prevent collisions and manage traffic flows. In response to increasing air traffic, a renewal process has been initiated. This includes deploying the automatic dependent surveillance-broadcast (ADS-B) communications protocol, which aims to enhance surveillance precision and increase the number of aircraft that can be handled simultaneously. This transition is transforming ATC from a radar-based system to a more advanced satellite-based global positioning system (GPS) location tracking system. However, due to its inherently open design, the ADS-B protocol lacks critical security features such as authentication, necessitating the adoption of additional security measures to mitigate potential cyber-attacks. To address these vulnerabilities, this work introduces Swarm, an innovative distributed ledger-based framework, built on top of the ADS-B protocol and aimed at enhancing the security of ATC while avoiding single points of failure. Swarm can be integrated into existing ATC infrastructure without requiring any modifications to the ADS-B protocol. We evaluate Swarm through rigorous and realistic attack scenarios, using real-world aviation data, demonstrating its capability to enhance the security of the aviation domain.

CCS Concepts: • **Networks** → **Location based services**; • **Security and privacy** → **Distributed systems security**;

We would like to express our gratitude to Professor Francesco Bruschi for his insightful advice on blockchain technology. This study was carried out within the MICS (Made in Italy–Circular and Sustainable) Extended Partnership and received funding from Next-Generation EU (Italian PNRR - M4 C2, Invest 1.3 - D.D. 1551.11-10-2022, PE00000004). CUP MICS D43C22003120001.

Authors' Contact Information: Gabriele Digregorio, Dipartimento di Elettronica, Informazione e Bioingegneria, Politecnico di Milano, Milano, Italy; e-mail: gabriele.digregorio@polimi.it; Edoardo Saputelli, Dipartimento di Elettronica, Informazione e Bioingegneria, Politecnico di Milano, Milano, Italy; e-mail: edoardo.saputelli@mail.polimi.it; Stefano Longari, Dipartimento di Elettronica, Informazione e Bioingegneria, Politecnico di Milano, Milano, Italy; e-mail: stefano.longari@polimi.it; Michele Carminati, Dipartimento di Elettronica, Informazione e Bioingegneria, Politecnico di Milano, Milan, Italy; e-mail: michele.carminati@polimi.it; Stefano Zanero, Dipartimento di Elettronica, Informazione e Bioingegneria, Politecnico di Milano, Milan, Italy; e-mail: stefano.zanero@polimi.it.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2025 Copyright held by the owner/author(s).

ACM 2471-2566/2025/09-ART44

<https://doi.org/10.1145/3759456>

Additional Key Words and Phrases: Distributed ledger, ADS-B, air traffic control (ATC)

ACM Reference Format:

Gabriele Digregorio, Edoardo Saputelli, Stefano Longari, Michele Carminati, and Stefano Zanero. 2025. Swarm: A Distributed Ledger-based Framework to Enhance Air Traffic Control Security Using ADS-B Protocol. *ACM Trans. Priv. Sec.* 28, 4, Article 44 (September 2025), 35 pages. <https://doi.org/10.1145/3759456>

1 Introduction

In the world of aviation, safety is a fundamental requirement. The air traffic control (ATC) service plays an essential role, with the primary purpose of preventing collisions (between aircraft, or between aircraft and obstructions on the maneuvering area of an aerodrome) and the secondary purpose of expediting and maintaining an orderly flow of air traffic [16]. Like any other technological field, the aviation domain has also received significant technical improvements over time. However, despite important advancements in ATC, the current airspace system infrastructure has not substantially changed since its last major developments in the 1970s. This period marked the beginning of what is known as the third ATC era, characterized by the introduction of digital radar automation, transponder-based surveillance, and early forms of data link communication [24]. This stagnation is in contrast to the constant growth in both passenger numbers and the number of aircraft in flight over the past decades, which creates new challenges [1, 45, 46].

A key component of the necessary process of renewal imposed by these new challenges is the automatic dependent surveillance-broadcast (ADS-B) protocol. It aims to transform the air transportation domain in terms of safety by increasing surveillance precision and allowing the ATC system to handle a greater number of aircraft simultaneously [23]. Moreover, it intends to transform ATC from a radar-based to a satellite-based global positioning system (GPS) location tracking system, enhancing the quality of traffic monitoring even in remote areas with low or even no radar coverage [32]. However, the openness-by-design of ADS-B, and its subsequent lack of authentication exposes the system to several types of attacks, threatening the integrity and trustworthiness of messages exchanged through the ADS-B protocol. Disruptions in ATC systems can have critical consequences, threatening human lives and impacting economic resources. Potential threat actors include state-sponsored groups aiming to cause geopolitical disruption, as well as terrorist organizations and politically motivated hacktivist groups. By exploiting the lack of authentication in ADS-B communications, an attacker could inject false aircraft data. In high-traffic airspace, particularly near major airports, such attacks may lead to delays, flight cancellations, and widespread operational confusion. Moreover, as general aviation pilots increasingly rely on ADS-B for situational awareness, such interference could also negatively influence their decision-making capabilities in flight.

Some solutions have been proposed to address the vulnerabilities inherent in the ADS-B protocol, aiming to enhance the security of such a critical domain. However, the analysis conducted by Manesh and Kaabouch [23] revealed significant limitations in mitigating these vulnerabilities. Firstly, none of the currently available solutions address all the potential security threats. Furthermore, many fail to consider their compatibility with the existing ADS-B infrastructure, which could result in prohibitive implementation costs due to the extensive and impractical changes required.

To address the identified challenges, our work presents an innovative approach aimed at enhancing the security of ADS-B systems without requiring extensive or impractical modifications to the existing infrastructure. With this aim, we developed Swarm, a decentralized framework based on a distributed ledger integrated with the ADS-B protocol to mitigate and potentially eliminate its current weaknesses. Swarm is designed to validate aircraft locations transmitted through ADS-B without necessitating a centralized source of truth, thereby avoiding the need for a

trusted third party and any single point of failure. Along with an exhaustive feasibility analysis, we also present a comprehensive validation strategy to set up the system configuration and tune the Swarm parameters. We assess the system's behavior under various parameter settings and evaluate its performance and security through a rigorous set of realistic tests, simulating diverse attack scenarios. This involves using historical air traffic data provided by the OpenSky Network, a non-profit community dedicated to collecting and sharing air traffic data for research purposes [26, 37]. The tests confirm Swarm's capability to counteract various threats and its potential to enhance security in the aviation sector.

The contributions of this work are the following:

- We introduce Swarm, a novel decentralized framework based on distributed ledger technology to enhance the security and integrity of data shared in ATC scenarios. This framework specifically addresses the most critical vulnerabilities of the ADS-B protocol while avoiding a single point of failure. We design Swarm to integrate seamlessly into existing ATC infrastructure without requiring modifications to the ADS-B protocol.
- Within Swarm, we design and integrate heuristics to validate the correctness of air traffic information before its inclusion in the distributed ledger, effectively mitigating a wide range of attacks against the system.
- We evaluate the feasibility, system configurations, and optimal parameter values of Swarm under realistic scenarios using real-world air traffic data. We also demonstrate the robustness of Swarm under exhaustive attack scenarios, confirming the Swarm's capability to enhance security in real-world aviation domains.

The article is structured as follows: In Section 2, we provide an overview of ADS-B and distributed ledger technologies, describing the current state of the art in both centralized and decentralized solutions for localization and aviation systems. In Section 3, we describe the motivation for our work, including an analysis of the threat model. In Section 4, we detail the approach applied during the development of Swarm. In Section 5, we conduct a feasibility analysis of the proposed approach and, in Section 6, justify some choices regarding Swarm parameters and settings. In Section 7, we evaluate the performance and effectiveness of Swarm under various realistic attack scenarios. In Section 8, we provide a proof of the formal security guarantees of our decentralized solution. Finally, in Section 9, we discuss about limitations of our work and, in Section 10, we conclude our work by outlining future research directions.

2 Background and Related Works

In this section, we introduce basic concepts related to the ADS-B protocol and distributed ledger technologies, along with an overview of existing state-of-the-art works that aim at finding centralized and decentralized solutions for localization and air traffic systems.

2.1 ADS-B Primer

The automatic dependent surveillance-broadcast (ADS-B) protocol is designed to allow air vehicles to both receive and transmit data in a broadcast mode, enhancing safety in air transportation by improving surveillance accuracy and facilitating the simultaneous management of a large number of aircraft [23]. The transmitted data can be captured by ground stations for surveillance purposes and by other vehicles. ADS-B is characterized by three key features: it is *automatic*, *dependent*, and *broadcast* [39]. It is *automatic* because no explicit stimuli are required for the protocol to operate; it is *dependent* because it relies on on-board systems to provide surveillance data to other parties; and it is *broadcast* because the data can be received by any entity, with the sender unaware at the time of transmission which entities will receive the data.

2.1.1 Technical Details and Data Content. Considered the satellite-based successor to secondary surveillance radar [37, 40], the ADS-B protocol leverages GPS location tracking to significantly enhance the quality of traffic monitoring, especially in remote areas where radar coverage may be minimal or nonexistent [32]. An ADS-B message contains various data elements including the aircraft's location and velocity, which are retrieved via global navigation satellite system (GNSS). It also includes the aircraft's identity and intent, urgencies, uncertainties, and additional information derived from onboard systems. While certain data elements, such as location, are broadcast periodically (e.g., twice per second), other types of information are event-driven [37].

2.1.2 Operational Role and Security Limitations. ADS-B is vital for achieving the performance objectives of the single european sky (SES), the next generation (NextGen) air transportation system, and India's aviation system block upgrade (ASBU), which include safety, capacity, efficiency, and environmental sustainability. It has already been implemented or is currently being implemented in regions such as North America, Europe, and the Asia/Pacific area [27, 39]. Additionally, ADS-B is recognized as part of the approved aviation surveillance technologies by the international civil aviation organization (ICAO) [10]. However, the absence of authentication in ADS-B communications leaves the system vulnerable to several types of attacks that, especially if the system further substitutes the secondary surveillance radar as the principal air surveillance protocol, could compromise the security and safety of the entire air transportation system. These potential attacks include flooding and jamming, target ghost injection, message injection, and message tampering, among others [24, 40].

2.1.3 Literature Solutions for Securing ADS-B. In the literature, several solutions have been proposed to enhance the security of ADS-B and address its security gaps [23]. Some approaches propose a paradigm shift for the ADS-B protocol, such as eliminating the openness of communication [15, 29, 31], which would require significant changes to the protocol's functionality.

HMAC-Based Protection. Viggiano et al. [41] proposed securing ADS-B messages using HMACs to ensure integrity and authenticity. However, their approach relies on fitting the HMAC into the message using "free bits" in the 1090ES (1090 MHz Extended Squitter) and **universal access transceiver (UAT)** formats. These are the two main data link technologies used in ADS-B: 1090ES is the standard for commercial aviation and operates on the 1090 MHz frequency, while UAT, operating on 978 MHz, is primarily used for general aviation in the United States. The 1090ES message format is tightly constrained, with a total of 112 bits—56 bits of payload and a 24-bit CRC—leaving no spare capacity for additional metadata. As a workaround, Viggiano suggested truncating the HMAC, which, however, compromises cryptographic strength and increases the risk of collisions.

ADS-Bsec: A Backward-Compatible Security Framework. Kacem et al. [18] addressed these limitations with ADS-Bsec, a holistic security framework that maintains backward compatibility by embedding HMAC metadata in place of the CRC fields. A 128-bit HMAC (e.g., using HMAC-MD5) is computed over the concatenated payloads of a pair of even and odd ADS-B messages. To fit this digest within the strict 112-bit ADS-B 1090ES message format, ADS-Bsec replaces the 24-bit CRC field in each message with a segment of the HMAC. These segments are derived through a custom bit-manipulation algorithm that compresses the original digest while retaining sufficient integrity protection. This approach allows cryptographic authentication without modifying the message structure or increasing bandwidth. In addition, ADS-Bsec integrates multilateration—a passive surveillance technique that estimates a signal's origin using time difference of arrival across synchronized ground receivers—to verify claimed aircraft positions and detect location spoofing. The framework also includes a "risk-adaptive cognitive engine" that uses environmental factors

(e.g., signal fading, interference) and rule-based inference to adapt its security posture dynamically. While ADS-Bsec avoids protocol-level changes, it depends on pre-distributed symmetric keys and secure key exchange mechanisms for both air-to-ground and air-to-air communication, which can introduce operational complexity—especially as aircraft cross jurisdictional boundaries with no shared trust infrastructure. Additionally, its multilateration-based verification depends on the availability of synchronized ATC centers, which may be sparse in remote airspace.

PKI-Based Approaches. Some solutions, moreover, focus on implementing public key infrastructures (PKI) to support ADS-B message exchange in a more secure manner. However, asymmetric encryption presents limitations when applied to ADS-B communication due to the need for a heavy encryption structure that would increase the message length and modify the ADS-B message format [23]. For example, in their work [28], Pan et al. propose a PKI-based scheme for ADS-B authentication. Nonetheless, this scheme introduces an unfeasible overhead for standard ADS-B communications, requiring five extra messages per transmission for the signature mechanism [18]. Despite these disadvantages, most of these PKI-based solutions are not able to guarantee and validate the correctness and soundness of the information contained in the shared messages, necessitating additional security mechanisms on top of the existing scheme. Costin and Francillon [12], on the other hand, suggested PKI-based solutions to mitigate message injection attacks. However, no implementations were proposed, making real-world deployment challenging.

2.2 Distribute Ledger Primer

Distributed ledger technology represents a decentralized solution for sharing information among participants, enabling the transfer of digital assets without any intermediaries [42]. Various distributed ledger paradigms have been proposed in the literature over time, each with different data models and technologies [2, 5, 13, 30]. The most widespread distributed ledger technology is the blockchain [4]. Introduced in the decentralized finance domain with the Bitcoin cryptocurrency in 2008 by Satoshi Nakamoto [25], the blockchain technology is appreciated for its features such as decentralization, persistency, pseudo-anonymity, and auditability [20, 49].

The blockchain consists of a chain of blocks, with each block unidirectionally linked to the previous one through a reference. The blockchain nodes are the only entities that contribute to adding new information to the shared ledger. This information is stored in the form of transactions, which represent the interactions occurring between the nodes. Blocks are collections of valid transactions accumulated over a period of time. In a blockchain, modifying any block would invalidate all subsequent ones, thereby ensuring the integrity of the chain.

To achieve agreement about the ledger's status among the distributed nodes, various consensus mechanisms have been proposed. The most prominent are proof of work (PoW) and proof of stake (PoS) algorithms. PoW requires nodes, known as miners, to solve complex mathematical puzzles, a process that is computationally intensive and consumes significant energy. The first miner to solve the puzzle earns the right to add the next block to the distributed ledger. This consensus algorithm is used by the Bitcoin cryptocurrency. Due to its high computational demand, this method has raised concerns about its environmental impact [35, 38]. Moreover, it is not suitable in scenarios where nodes have limited computational resources. PoS, on the other hand, offers a less resource-intensive alternative [34], where the creator of a new block is chosen based on their stake. This stake could be, for instance, the amount of tokens a node owns.

Early work on decentralized trust mechanisms has laid the groundwork for modern reputation-based consensus models. A notable example is the Karma system [8], which introduced a reputation-based approach for peer-to-peer overlays. In Karma, each node is assigned a uniformly random identifier, and neighbors are selected independently of trust assumptions, based solely on the

identifier space. This randomization strategy supports fairness and load balancing while resisting collusion and centralization—principles that directly mirror modern blockchain sharding and validator assignment mechanisms. Swarm builds on similar foundations by leveraging local validation and global reputation scores to create a trustless, self-regulating network that does not depend on centralized control or static infrastructure.

2.2.1 Distributed Ledger-Based Localization Systems. Different localization solutions have been proposed that leverage the unique features of distributed ledger technology to enhance security and integrity. These solutions demonstrate the potential of distributed ledgers in improving localization systems through secure and decentralized features. Three notable examples of such approaches are the internet of entities [33], a solution designed to provide a secure and reliable localization system for people and things; BlockLoc [7], a localization system tailored specifically for the internet of things (IoT) domain that uses blockchain technology to ensure the integrity and authenticity of localization information; and Goliath [22], a blockchain-based framework aimed at implementing a decentralized system to monitor the localization of entities within the automotive field.

2.2.2 Distributed Ledgers in the Aviation Domain. Pennapareddy and Natarajan [29] provided a comprehensive survey on the application of blockchain technology in the aviation industry, covering various use cases and proposing their solution based on a dynamic flight planning system.

Blockchain in UAS Traffic Management. Allouch et al. proposed UTM-Chain [3], a blockchain-based framework for unmanned aircraft system traffic management, leveraging Hyperledger Fabric for secure flight data storage and decentralized identity management. Similarly, Baptista et al. introduced DFLy [6], a publicly auditable and privacy-preserving UAS traffic management system, using Ethereum and zero-knowledge proofs to ensure secure registration and flight authorization. However, while both solutions enhance unmanned aircraft system security and compliance, they are primarily designed for low-altitude drone operations.

Distributed Ledgers for ATM Security. To enhance the security and trustworthiness of air traffic management (ATM), a variety of distributed ledger systems have been proposed, each addressing different aspects of the aviation ecosystem. These works highlight the potential impact of distributed ledger technology in the aviation sector, addressing its security challenges and establishing trustworthiness in such systems. For instance, Lu et al. [21] proposed ATMChain, an information security framework for ATM based on an alliance blockchain. Their work introduces a unique consensus mechanism and a hierarchical data storage structure to enable secure, multi-center, and traceable data sharing and management within the ATM system. Clementi et al. [11] introduce a high-level, blockchain-based conceptual framework for securing the sharing of flight data. Their work details a layered architecture that decentralizes the currently centralized flight planning process, providing a distributed storage platform to ensure data integrity, traceability, and availability.

Other research efforts have been conducted to enhance the ATC context, leveraging the features offered by the Ethereum blockchain. In one approach, Hasin et al. [15] specifically address the security of ADS-B based ATM systems by proposing an architecture built on a private Ethereum blockchain. Their model incorporates aircraft and various ground components as permissioned nodes, using smart contracts to manage policies and secure otherwise unencrypted data links. Similarly, Wu and Liu [44] present a trusted authentication model that combines an Ethereum blockchain with the **interplanetary file system (IPFS)** to secure ground-to-air communications. In their architecture, IPFS stores detailed device information off-chain to reduce latency and bandwidth consumption, while Ethereum smart contracts manage the registration and certification of avionics to create a trusted domain. This ensures that all communication is verified through a combination of digital signatures and certificate checks. Finally, Wu et al. [43] presented BlockTrust,

a blockchain-based trust model for ATM networks. Their work introduces a three-layer architecture (user, broadcasting, and terminal) and a hierarchical **proof of level (PoL)** consensus mechanism.

2.2.3 Limitations of the Existing Solutions. Existing solutions that rely on message authentication, location verification mechanisms, or blockchain-based approaches exhibit significant limitations in addressing air traffic vulnerabilities. These solutions often fail to comprehensively mitigate critical security threats, particularly those related to the trustworthiness of data declared by air vehicles.

For example, solutions centered on message authentication, such as the one proposed by Wu and Liu [44], can introduce significant complexity and potential latency by relying on multi-step verification processes across external systems.

Many blockchain-based proposals, while enhancing integrity, have their own shortcomings. Some are purely conceptual and lack the validation of a concrete implementation [11], while others acknowledge their cryptographic approach inevitably introduces communication delays and overhead [43]. Furthermore, several models rely on private or permissioned architectures that require trusted authorities for governance and key generation, partially re-introducing the centralization they aim to avoid [15, 21]. A critical, shared vulnerability is that these systems often focus on ensuring data cannot be tampered with after it is written to the ledger; they lack robust mechanisms to verify the correctness and trustworthiness of the air traffic data before it is immutably recorded.

Attacks against distributed ledgers. While distributed ledgers provide a decentralized and immutable solution for sharing information, different attacks that might compromise the entire system have been suggested and studied in the literature. Attackers could target key components of the distributed ledger: the transaction level, the block level, and the consensus level [22]. At the transaction level, attackers generate fake transactions or flood the system with spam. At the block level, they attempt to modify blocks. At the consensus level, malicious entities try to gain a majority of the consensus power to control the distributed ledger.

These attacks are often overlooked in current research applying distributed ledger technology to the aviation domain, which could pose serious threats to air traffic. For instance, some research presents high-level conceptual frameworks that explicitly defer detailed security analysis of such attacks as future work [11]. Other works focus primarily on using the ledger for application-level security—such as authenticating devices or securing data channels—without providing a rigorous experimental validation of the underlying distributed ledger against sophisticated transaction, block, and consensus-level attacks [15, 44]. Even when consensus security is considered, it is often a high-level theoretical analysis against collusion rather than a practical test against targeted manipulation, or the design of the consensus mechanism itself is considered out of scope [43].

2.2.4 Proposed Solution. In response, we introduce Swarm, a distributed ledger-based solution that ensures decentralized trust management and anomaly detection for shared air traffic information. Swarm effectively addresses the most critical vulnerabilities of the ADS-B protocol by incorporating heuristics to verify air traffic data before integrating it into the distributed ledger, thereby mitigating a wide range of potential attacks and ensuring data correctness and trustworthiness. Furthermore, we have designed Swarm to be resilient against blockchain-level threats, rigorously testing it against various attack vectors at the transaction, block, and consensus levels. Unlike traditional solutions, Swarm does not rely on key management and centralized entities, enabling a scalable and adaptive security mechanism for air traffic surveillance. Our approach requires no modifications to the ADS-B protocol and does not introduce any overhead to classical ADS-B communication and message exchange. Instead, it operates as a complementary system that passively utilizes ADS-B messages without interfering with their normal transmission. Swarm

is designed to support standard ADS-B operations by monitoring the coherence and validity of exchanged messages, discarding information that fails integrity and trustworthiness checks.

3 Motivations and Threat Model

The importance of air traffic data security derives from the delicate nature of the aviation domain and its critical assets. The primary purposes of ATC services are to prevent collisions and maintain an orderly flow of air traffic [16]. Therefore, any disruption or failure in ATC systems could have severe consequences. The involved assets include the safety and lives of people as well as the valuable economic resources at stake.

The presented work focuses on the ADS-B protocol due to its key role in next-generation traffic control technologies. ADS-B is not authenticated, exposing it to potential threats that can be categorized into service disruption and data manipulation [40]. Service disruption can occur through a denial of service (DoS) attack by flooding the system with a high volume of fake data, causing an overload that compromises its ability to provide normal ATC services. Data manipulation, on the other hand, aims to alter the system's results and decisions by sending malicious data as legitimate, leading to its acceptance. Such issues could, without appropriate mitigations at a system, procedural, or digital level, compromise the availability and efficiency of transportation systems, or even in extreme cases jeopardize safety.

In this work, we aim at developing a distributed ledger-based decentralized solution to enhance the security and integrity of data exchanged within the aviation domain. Our objective is to verify the correctness and soundness of air traffic information before its inclusion in the distributed ledger, thereby effectively mitigating potential attacks. In our vision, we do not consider airports necessarily trustworthy entities but instead implement a fully decentralized system in which both airports and aircraft can earn and lose reputation while sharing their observations. This approach leads to a democratic decision-making process regarding the inclusion or rejection of information in the distributed ledger. The system is designed to prevent single points of failure and centralized decision-making. Additionally, it enables information validation in areas where no airports are available or where existing airports may not be trustworthy due to geopolitical or strategic reasons. It is important to note that our objective does not focus on emergency situations that require urgent, short-term responses. Instead, it focuses on continuous tracking over time to detect anomalies.

To better define the risks that ATC systems face, in the following, we build a succinct threat model, reviewing threat agents potentially interested in attacking the system, the attack surfaces they might exploit, and the potential attacks they could execute with this aim.

Threat Agents. Among the potential threat agents aiming to compromise the ATC system, our opinion is that the most significant ones are not financially motivated. State-sponsored attackers with an interest in disrupting services for geopolitical reasons are surely at the top of the list (witness the ongoing stream of attacks against GNSS systems and their impact on aviation [36]). Other potential threats could be terrorist organizations or more benign hacktivist groups, aiming to make political statements.

Attackers Goals. Most likely, the type of attackers we identified will aim to disrupt regular air traffic control services, resulting in a DoS that could impede the orderly flow of traffic, and potentially escalate to compromise operational safety. By sending malicious ADS-B communications, the attacker could simulate the presence of non-existent aircraft in the airspace, overloading controllers and decreasing their situational awareness. The effect of confusion would be increased by the impossibility of establishing communications with the “ghost” aircraft. In busy airspace around major airports, this disruption, if targeted to approach controllers, could lead to delays and cancellations. Additionally, in particular on board general aviation aircraft, ADS-B information is

used by pilots for direct situational awareness, making it possible for an attacker to also impact their decision-making processes.

Attacker capabilities. Attackers need relatively limited resources. They basically need an ADS-B transponder (which can be easily implemented through a software-defined radio (SDR), for instance) in order to transmit spoofed information. Attacker must be within the communication range afforded by their equipment. The increasing accessibility of the technology necessary for such attacks further amplifies these risks, making it imperative to understand and mitigate potential threats to ensure the safety and integrity of ATC operations.

4 Swarm Approach

Our goal is to implement a decentralized blockchain-based system built on top of the ADS-B protocol that can monitor aircraft location without requiring a centralized source of truth. We select the Solana blockchain as a suitable solution for managing the high data demands of our framework. Founded by Anatoly Yakovenko, Solana aims to achieve high performance without compromising decentralization or security. Indeed, it offers a higher transactions per second (tps) rate compared to other distributed ledger solutions, making it compatible with the frequency of ADS-B message transmissions [47, 48]. The Solana blockchain integrates the PoS consensus mechanism and the proof of history (PoH) technique. PoH provides a decentralized, verifiable, and secure timestamp for network transactions, enabling nodes to agree on the sequence of events without relying on external timestamps and effectively addressing the challenge of transaction ordering in a trustless environment. An overview of the system is depicted in Figure 1.

We proceed to describe Swarm by its main components: the distributed ledger data structure, the node types, and the consensus mechanism.

4.1 Distributed Ledger Data Structure

Both the interactions between nodes and the concept of a transaction within our framework are dependent on how information is exchanged using the ADS-B protocol. The ADS-B protocol employs a broadcast method for sending messages, which are then received by all potential receivers within the sender's communication range. Based on this, we formalize the interaction between two nodes at a given timestamp as the exchange of ADS-B messages between them. Additionally, in our system, we introduce the concept of an ADS-B snapshot: a summary of all information sent by a specific aircraft during a given time frame through the ADS-B protocol. In the proposed distributed ledger system, each transaction is generated based on these ADS-B snapshots. By aggregating all the messages received during a certain timestamp (i.e., one second) into an ADS-B snapshot rather than generating a transaction for every single message, we significantly reduce the volume of transactions generated and data to process and store for each node in the system.

Each transaction in our system is composed of the following elements:

- The unique identifier of the target node.
- The unique identifier of the sender node.
- The ADS-B snapshot representing the interaction between the two nodes.
- The target's location determined using GNSS system.
- The timestamp indicating the exact moment of the interaction.
- The digital signature of the target node, which is always the one originating the transaction, required by the network to verify the transaction's authenticity and integrity.

After an interaction occurs between two nodes via ADS-B communication and a transaction is generated, the transaction is shared with the other distributed ledger nodes. This sharing step occurs through a different means than the ADS-B protocol, in accordance with the network requirements

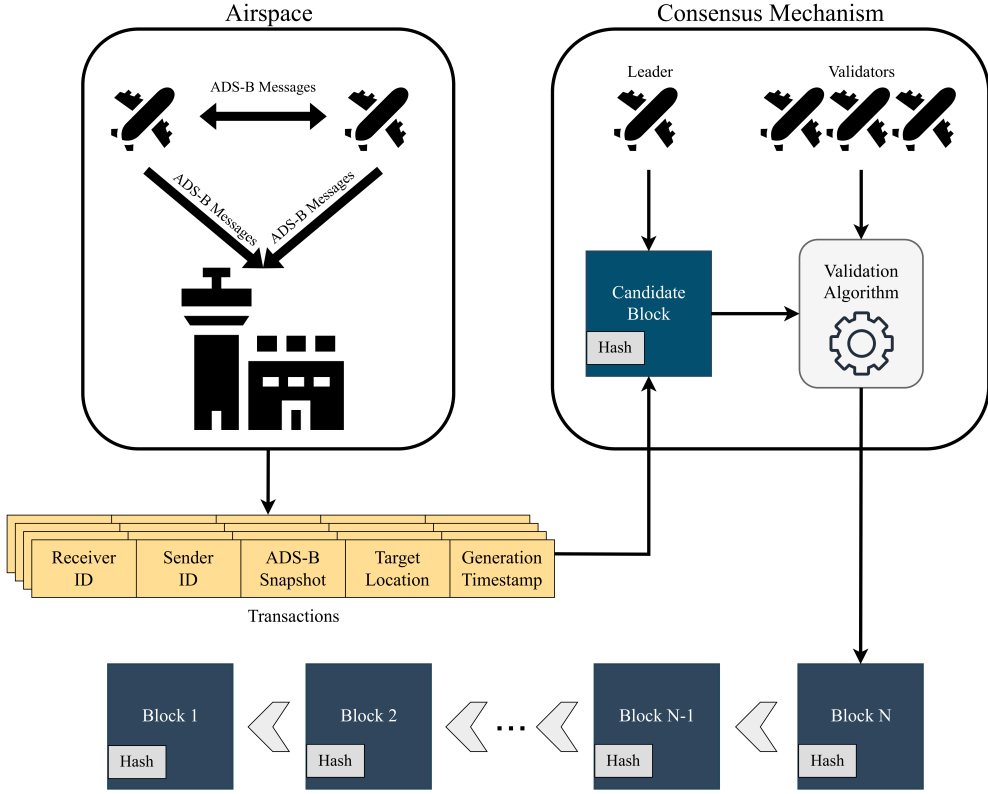


Fig. 1. Block creation process within Swarm. It begins with node interactions, continues with the creation of the transactions, and ends with the transactions' validity check performed through the *Validation Algorithm*.

of PoS systems, which typically rely on a peer-to-peer network. This can be achieved using cellular or satellite networks. The transaction is then evaluated and if deemed legitimate, it is added to a new block of the distributed ledger. Each block contains up to a maximum number of transactions, defined by the block-size value, which are added to the distributed ledger after being validated through the consensus mechanism and, more specifically, by its validation algorithm component, presented in Section 4.3.1. A new block is inserted in the distributed ledger after a fixed amount of time from the previous insertion, defined by the block-time value.

As shown in Figure 2, the blocks contain all the information that defines the current state of the distributed ledger and each of them references the previous one through its hash value, forming a chained structure. Moreover, each block contains a series of information about itself, such as the block height, the timestamp, and the attempt number. The attempt number is used to distinguish between different attempts to create a block when the consensus mechanism fails, resulting in the block's rejection. Following this, the block lists the valid transactions that are part of it and provides information about each participant, including their identifier in the form of a 24-bit ICAO address or a 3-letter international air transport association (IATA) code, their reputation, and an activity flag that indicates whether the participant has been involved in at least one of the transactions included in the block. Additionally, a record of the identifiers of the last N entities with which each participant has interacted is included. As explained in Section 4.3.2, including this record

Block Height				
Generation Timestamp				
Previous Block Hash				
Attempt Number				
Receiver ID	Sender ID	ADS-B Snapshot	Target Location	Generation Timestamp
...				
Receiver ID	Sender ID	ADS-B Snapshot	Target Location	Generation Timestamp
Node ID	Reputation	Activity Flag	Last N Interactions	
...				
Node ID	Reputation	Activity Flag	Last N Interactions	
Leader Signature				
Current Block Hash				
PoH Hash 1				
...				
PoH Hash K				

Fig. 2. Structure of the distributed ledger blocks used in the proposed framework.

is necessary to mitigate reputation boost attacks. Finally, the block concludes with the digital signature of the node that generated it (i.e., the leader), the hash of the block, and the PoH hashes. In the PoH scheme, the block does not contain the entire sequence of hashes; instead, it includes only a subset, which is sufficient for validators to assess the validity of the block.

A final note concerns resource optimization, which is particularly relevant for embedded systems such as those in smaller airplanes. These systems often have limited memory, which poses a critical issue as the amount of memory required to store the complete ledger continuously grows. Following the approach described in [17, 19, 22], we employ a state blocks-based method. This method involves assigning the task of storing the complete ledger to participants with more memory resources, such as airport nodes in our context. When a sufficient number of regular blocks have been produced, the consensus mechanism aggregates these into what is termed a state block. This process allows participants with limited storage capacity to maintain only the most recent state block and any subsequent regular blocks. Additionally, this approach simplifies the process for new nodes joining the distributed ledger, as they need to download and verify less data.

4.2 System Participants

Types of Entities. Two categories of entities participate in Swarm: aircraft and airports. While aircraft are represented by their 24-bit ICAO address, airports are identified through their 3-letter IATA code. However, only aircraft can send ADS-B messages, while both aircraft and airports can receive ADS-B data. This means that, in a transaction, the identifier of the sender will always be filled with the ICAO code of an aircraft, while one of the targets can either be an aircraft ICAO code or an airport IATA code.

Role of Aircraft. Aircraft nodes generate ADS-B messages, collect ADS-B messages from other nodes in snapshots, and broadcast transactions.

Role of Airports. Airports contribute to various aspects of the system. Their first role is in the creation of the genesis block, i.e., the first block of the distributed ledger. This block must serve as a valid and robust starting point for the ledger. However, during this phase, the node reputation system has not yet been initialized, necessitating different criteria for selecting nodes that can generate trustworthy transactions to be included in the genesis block.

Trust Assumptions in Initialization. Although there are no technical constraints or system limitations preventing the selection of aircraft nodes for this task, we start from the reasonable assumption that airports are more likely to be trustworthy than aircraft. For this reason, we designate a subset of airports as trusted entities responsible for generating the transactions to be included in the genesis block. A possible criterion for selecting trustworthy airports in a real scenario could be based on their operational and security standards. Additionally, the selection process could consider factors such as airport traffic volume, regulatory compliance, and historical reliability. It is important to emphasize that this trust applies only during the initialization phase of the system, while long-term risks are mitigated through the trust evolution mechanism of the reputation-based PoS system on which Swarm's security model is built. Once the genesis block has been created, each selected airport will be treated like any other entity and will have the ability to gain or lose reputation based on the validity of the transactions it generates.

4.3 Consensus Mechanism

A decentralized framework requires a consensus mechanism to reach agreements among the distributed nodes of the system. This mechanism is composed of two key components: the validation algorithm and the consensus algorithm. The validation algorithm aims to preserve the integrity and accuracy of the system by verifying the validity of transactions added to the ledger, thereby mitigating transaction-level attacks. The consensus algorithm determines how distributed nodes reach an agreement on the state of the distributed ledger, thereby providing security at both the block and consensus levels.

4.3.1 Validation Algorithm. Following other trust-based approaches [22], Swarm considers a transaction legitimate only if it complies with specific heuristics. We have designed three heuristics: *Illegal Transaction Filtering*, *Location Validation*, and *Ghost Aircraft Penalization*.

Illegal Transaction Filtering. It filters out illegal transactions in which both the sender and the target node have identical identifiers. This helps to avoid transactions aimed at self-boosting the reputation of an attacker, thereby protecting the system's integrity at the transaction level.

Location Validation. It marks as invalid all transactions containing aircraft locations that are not physically realistic or do not comply with those stored in previous blocks of the distributed ledger. Each target node is aware of its own position, as well as that of the sender. Therefore, it is possible to compute their relative distance and only generate the transaction if it falls within the generation range established by the system. Any transaction involving nodes located at a distance greater than the transaction generation range is marked as invalid. The choice of this range depends on the ADS-B protocol communication range and is discussed in Section 5.3.2. If the transaction satisfies the distance constraints imposed by the generation range, the *Location Validation* heuristic also verifies the sender and receiver locations declared in this transaction against the locations stored in the previous LV_{Blocks} blocks of the distributed ledger, where LV_{Blocks} is a parameter determined at implementation time. The validation process considers the maximum velocity v_{max} at which an aircraft can fly to compute the upper bound of the aircraft's travel distance based on the time interval Δt elapsed between the current transaction and each previously stored one. Each location used for this evaluation is weighted directly proportional to the reputation of the node that generated the transaction containing this information.

Ghost Aircraft Penalization. It helps address the issue of false location claims made by potentially malicious entities. The *Ghost Aircraft Penalization* validates the location of a node declared in a transaction by ensuring that other entities within the transaction generation range of this node have received one or more ADS-B messages from the node at that location. However, expecting confirmation from *all* entities within the transaction generation range of a specific node is not

feasible and could even be harmful to the system. An attacker might attempt to disrupt the system by falsely claiming not to have seen a specific entity, or an ADS-B message might not be received by all nearby entities due to external factors like atmospheric conditions or the Earth's geological conformation. Therefore, the *Ghost Aircraft Penalization* heuristic requires only that a sufficient number of entities within the generation range confirm the location information for a transaction to be considered valid. Similar to the *Location Validation* heuristic, the *Ghost Aircraft Penalization* heuristic weights the value of each declaration used to evaluate a transaction according to the reputation of the node that generated the transaction storing the information.

4.3.2 Consensus Algorithm. Taking inspiration from Goliath [22], we develop our distributed ledger system based on a reputation-based PoS consensus algorithm, a resource-efficient solution. In a PoS consensus algorithm, the ability of a node to be elected as a leader or validator—meaning its ability to participate in the key steps of the consensus mechanism—is proportional to its stake. In Swarm, the stake of a specific node corresponds to its reputation, which depends on the behavior of that node. If a node demonstrates a good attitude and contributes to the proper functioning of the system, its reputation increases, hence increasing the likelihood of being entrusted during block generation and validation. Conversely, if its actions are considered malicious or harmful to the system, its reputation decreases, excluding the node from critical system operations.

Reputation Update Mechanism. Each time a new block is created and added to the distributed ledger, at intervals known as block-time, the chosen block leader evaluates and updates the reputation of each node involved in generating at least one transaction in the block, based on the validity of the transactions in which they have been involved. The reputation of each active node i in block B is updated by retrieving its current reputation score from the previous block and adjusting it following the formula

$$Rep_i = oldRep_i + accTr_{i,B} \cdot R_{Val} \cdot SF_{i,B} - rejTr_{i,B} \cdot P_{Val}$$

Formula Terms.

- $accTr_{i,B}$ (accepted transactions) represents the number of transactions generated during the block-time interval for block B that involve node i and have been marked as valid by all the heuristics included in the validation algorithm.
- $rejTr_{i,B}$ (rejected transactions) represents the number of transactions generated during the block-time interval for block B that involve node i and have been marked as invalid by at least one of the heuristics included in the validation algorithm.
- R_{Val} and P_{Val} represent the factors used for increasing and decreasing the reputation of active entities, respectively.
- $SF_{i,B}$ represents the scaling factor, used to mitigate the capability of groups of malicious participants to boost each other's reputation. This type of attack is discussed in Section 7.2. The scaling factor value for node i and block B is computed as

$$SF_{i,B} = \frac{|new_{i,B} - K_i|}{new_{i,B}}$$

where $new_{i,B}$ is the set of participants interacting with node i according to the transactions stored in block B , and K_i is the set containing the last N entities that have interacted with node i . N is chosen according to the realistic maximum number of interacting attackers that the system is expected to handle.

Bounding Stake to Avoid Centralization. In classic and well-known PoS applications, such as Ethereum, the stake is unbounded and directly correlated with monetary value. For a node or

group of nodes, acquiring a stake large enough to significantly increase their probability of being selected as validators—and thus raise concerns about system security—would require an unfeasibly large amount of money. In Swarm, although we refer to “stake” for consistency, it is not linked to economic value but rather to the trust that the system places in a node’s behavior over time. Allowing a node to accumulate stake without limits would pose a security risk in the Swarm scenario, as certain nodes could gain an excessively high stake over time. This is particularly relevant for airports, which are never-offline participants constantly involved in air traffic operations. Over the long term, this would create a significant discrepancy between less active or newly added nodes and airports, granting the latter an unfair advantage in the validation process—an advantage that could be exploited by attackers. Specifically, the stake of a limited number of nodes—primarily the airports—would become unboundedly high over time, eventually reaching a level that allows them to take full control of the system in the long run.

Maximum Reputation Limit. In the Swarm setting, we impose a bound on the stake to ensure sufficient long-term homogeneity among well-behaving nodes. This mechanism maintains a high turnover among validators, preventing a few nodes from monopolizing control over the distributed ledger. Swarm sets a maximum reputation score of 4,096, while the minimum value is set to 0. Nodes with a score of 0 are not allowed to participate in the consensus mechanism. In our experimental settings—described in Sections 6 and 7—we observe the long-term distribution of stake among nodes. Our results confirm the tendency toward a stable equilibrium among nodes with a sufficiently high stake. This ensures that no node or group of nodes has an excessively high probability of being elected as a validator.

Reputation Decay for Inactive Nodes. Moreover, Swarm incorporates a mechanism that gradually reduces the reputation of inactive nodes based on two parameters: IN_{Blocks} and IN_{Factor} . IN_{Blocks} determines the time period after which the reputation of inactive nodes starts to decrease, while IN_{Factor} defines the rate of decrease. This approach emphasizes the importance of recent transactions over older ones. Optimal values of the decreasing factors for inactive nodes, alongside R_{Val} and P_{Val} , are discussed in Section 6.

4.3.3 Leader Integrity Check. During block creation, the available transactions are included in a block, including those proposed by the leader. The leader is discouraged from adding misleading information due to the combined influence of the reputation-based PoS system and the validation algorithm. Since the leader is selected based on its reputation, it is likely to have a high reputation due to past legitimate behavior. However, even if a malicious leader has gained high stakes through previous legitimate actions, it cannot arbitrarily insert incorrect information. Any information it includes in the block must be consistent with data reported by other nodes to pass the validation process. The validation algorithm ensures that all included transactions adhere to physical and logical constraints and is enforced by validators selected based on their stake amount. If the leader attempts to introduce false information that contradicts the observations of other participants, the block will fail validation. In such cases, the transactions are rejected, and the leader’s reputation score is reduced, decreasing its likelihood of being selected for future block generation. The robustness of this approach is enforced by the decentralized nature of the system. While Swarm does not explicitly penalize leaders for omitting valid transactions, the short block-time and decentralized transaction propagation ensure that such transactions are included in subsequent blocks. Indeed, due to the random, stake-based leader selection in the distributed ledger, it is infeasible for an attacker to control consecutive leaders. This makes coordinated omission attacks impractical, as no single leader can reliably suppress transactions over time.

Table 1. Size in Bytes of Each Block Field

Size (Bytes)	Block Field	Size (Bytes)	Block Field
4	Block Height	4	Generation Timestamp
32	Current Block Hash	32	Previous Block Hash
$222 \cdot N_T$	Transactions	4	Attempt Number
$3 \cdot N_P$	Participant IDs	$8 \cdot N_P$	Participant Reputations
$3 \cdot N \cdot N_P$	Participant Interactions	N_P	Participant Activity Flags
128	Block's RSA Signature	$32 \cdot N_H$	Proof of History Hashes

Table 2. Size in Bytes of Each Transaction Field

Size (Bytes)	Transaction Field	Size (Bytes)	Transaction Field
3	Sender's ICAO code	3	Target's ICAO/IATA code
8	ADS-B latitude	8	ADS-B longitude
8	ADS-B velocity	8	ADS-B heading
8	ADS-B vertical rate	7	ADS-B callsign
1	ADS-B on-ground flag	1	ADS-B spi flag
2	ADS-B squawk code	8	ADS-B barometric altitude
8	ADS-B geometric altitude	8	Sender's latitude
8	Sender's longitude	4	Timestamp
128	Sender's RSA signature		

5 Feasibility Analysis and System Configurations

To develop a framework suitable for real-world implementation, a feasibility analysis of Swarm is conducted. We estimate both the memory requirements of the proposed distributed ledger-based framework and the system's ability to handle transaction volumes that align with realistic aircraft in-flight numbers.

5.1 Memory Requirements

Table 1 presents the memory requirements of each field of a distributed ledger's block, as detailed in Section 4.1. The core component of each block in the distributed ledger is represented by the transactions stored within it. The number of transactions per block, denoted by N_T , depends on the transaction volume generated at a given timestamp. Table 2 details the memory requirements for each field of a transaction. According to this data, each transaction requires 222 bytes of storage, leading to a total memory requirement of $222 \cdot N_T$ bytes per block. Additionally, each block stores the reputation value, the activity flag, and the list of the last N interactions for each participant. N_P represents the number of participants in the block. Furthermore, N_H , the number of PoH hashes stored in a block, must be considered. Each PoH hash, being a SHA-256 hash, requires 32 bytes. We assume N_H to be $\frac{1}{100}$ of the number of transactions N_T .

5.2 Computational Complexity

In this section, we provide a high-level complexity analysis of the main algorithmic components of the Swarm framework.

5.2.1 Snapshot Generation. Each aircraft, at each timestamp of 1 second, collects ADS-B messages and summarizes them into an ADS-B snapshot. Let k be the number of ADS-B messages received

in a timestamp from a single sender. The complexity of aggregating these messages is $O(k)$. In practice, k is small and limited to the ADS-B message frequency; hence, this phase does not pose a significant computational overhead.

5.2.2 Validation Algorithm. This is the set of heuristics for deciding whether a transaction is valid before placing it into a block. We discuss each heuristic separately and then compute an aggregated complexity that takes into account all the algorithms described in Section 4.3.1.

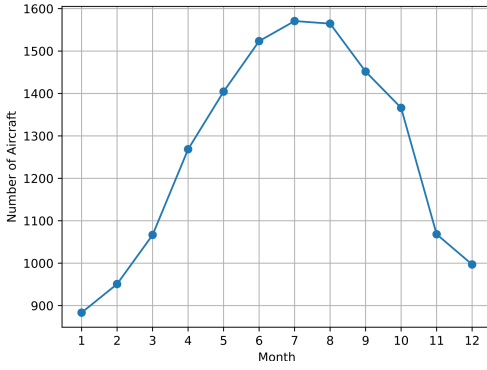
- **Illegal Transaction Filtering:** Checks if the sender and receiver have the same ID. Hence, the check is $O(1)$ for each transaction.
- **Location Validation:** Ensures each claimed position is consistent with physically possible flight distance, comparing new locations with locations from the last LV_{Blocks} blocks. Let's suppose ℓ is the number of prior records from LV_{Blocks} blocks we need to compare to. Checking each location involves a distance check (constant time) and a velocity/time feasibility check (also constant time). So the complexity is $O(\ell)$ per transaction. In practice, LV_{Blocks} is very small and, hence, ℓ is small too, so this step remains efficient.
- **Ghost Aircraft Penalization:** Ensures other nearby nodes confirm the existence of the aircraft at the claimed position. If m nodes are within the generation range, the system checks whether enough of them have declared seeing this aircraft. Verifying each node's record is an implementation detail. In the worst case, for each node, given σ snapshots to check, we need to scan all of them, hence with a complexity of $O(\sigma)$. However, if we assume a minimal hashtable structure, we can assume a single lookup in constant time on average, hence $O(1)$. We consider the constant time case, for simplicity. This step is repeated m times, one for each neighbor, with a final complexity of $O(m)$. The approach weights each statement by the node's reputation, adding at most a constant factor. In practice, m is small enough to have a low computational complexity.

Aggregating these values, each heuristic operates in at most $O(\ell + m)$ per transaction, with ℓ and m both relatively small in real-world scenarios.

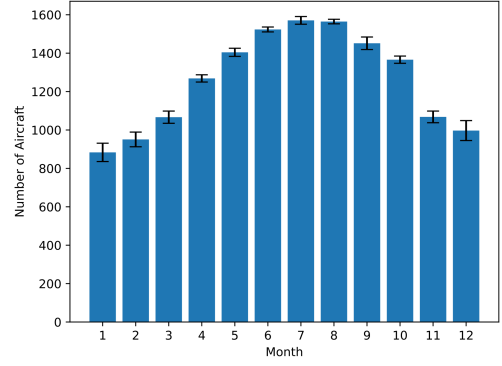
5.2.3 Consensus Algorithm. Swarm uses a reputation-based PoS to select leaders, where each node has a reputation score that determines its likelihood of becoming a leader. The system chooses a leader from all participants, weighting each node by its reputation, from a distribution of, let's say, N nodes. The naive approach takes $O(N)$, while techniques such as alias tables or binary search over a prefix-sum array reduce sampling to $O(\log N)$. At each round, the leader node collects valid transactions and forms a block. Let T be the number of transactions being included in the new block. The block leader runs the validation checks on each transaction. From above, each transaction check is $O(\ell + m)$, hence creating a block has complexity $O(T(\ell + m))$. On the other hand, signature checks to verify the node's digital signature can usually be done in $O(1)$ or $O(\log(\text{key size}))$ time, depending on the cryptographic scheme.

Unlike PoW systems, in PoS each block is hashed exactly once during finalization. While the exact performance depends on the hash algorithm chosen, we can assume a linear time complexity with respect to the total size of the data being hashed. Given $|B|$ as the size of the block, the complexity is $O(|B|)$. Once a block has been created, the leader updates the reputation of each node involved. This requires passing over the nodes with accepted or rejected transactions in that block. If P nodes participated in that block, the updates take $O(P)$.

5.2.4 State Block Generation. Swarm uses a state block approach to keep memory usage reasonable. Periodically, a special state block aggregates past blocks. Generating a state block mainly involves aggregating the ledger state. Suppose there are T' total transactions from the past set of blocks. Summarizing them is $O(T')$ to form the new state. However, this operation is infrequent and



(a) Line plot of the monthly average number of aircraft in flight.



(b) Bar plot of the monthly average number of aircraft in flight, with a 95% confidence interval.

Fig. 3. Monthly average number of aircraft in flight in the European airspace throughout the year 2022, according to the OpenSky Network database [26, 37].

can be handled by nodes with more resources (e.g., airports); hence, its impact can be considered easily amortizable.

5.3 Data Volume

As explained in Section 4.1, a transaction captures the interaction between two nodes. Specifically, at each timestamp, the target node collects all ADS-B messages sent by the sender node and creates an ADS-B snapshot that summarizes this information. The target node generates a transaction based on each ADS-B snapshot, digitally signs it, and broadcasts it to the network. Thus, each target node generates a number of transactions equal to the number of sender nodes within a fixed range for that specific timestamp. Considering the number of sender nodes for each target node results in the same number of generated transactions as considering the vice-versa. For convenience and simplicity, we consider the former approach for our evaluation, obtaining the formula

$$T_t = A_t \cdot R_t$$

where:

- T_t is the number of transactions generated during the timestamp t .
- A_t is the number of aircraft in flight during the timestamp t .
- R_t is the average number of target nodes within the transaction generation range of a sender node during the timestamp t .

By estimating the average values of A_t and R_t , along with the block-time and block-size, we gain insights into designing a feasible and realistic implementation from a data usage perspective.

5.3.1 Number of Aircraft in Flight. For a realistic estimation of the parameter A_t , we rely on historical data retrieved from the OpenSky Network database [26, 37]. This involves collecting real-world data about the number of aircraft in flight each day throughout the year 2022. We sample this data once per hour within the chosen area, covering the vast majority of European airspace defined by latitudes between 35 and 72 degrees and longitudes between -12 and 40 degrees. We aggregate this data to compute daily, monthly, and annual averages. These steps produce an average daily number of aircraft in flight of approximately 1260.65, with a 95% confidence interval ranging from 1235.04 to 1287.85. The results are also depicted in Figure 3, which shows significant

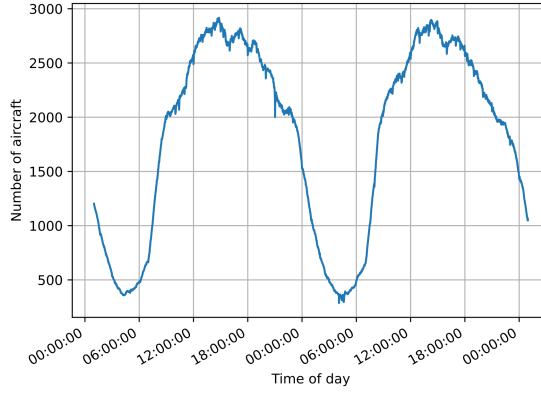


Fig. 4. Number of aircraft flying in the European airspace between October 2, 2022, and October 4, 2022, according to the OpenSky Network database [26, 37].

differences between the months of the year. Notably, the summer months report a higher number of aircraft compared to the winter months, largely due to holiday travel.

5.3.2 Number of Receivers. Similar to the estimation of the parameter A_t , to estimate R_t we rely on historical data from the OpenSky Network database. However, the computation of R_t 's average value for the entire year of 2022 is not feasible due to excessive computational demands and time constraints. Therefore, to estimate the average value of R_t , we rely on flight data from an intermediate season, which represents an average scenario in terms of flight numbers. We select data from European airspace during the period from October 2, 2022, at 00:00:00, to October 4, 2022, at 00:00:00. The extracted data are shown in Figure 4. This figure depicts a significant decrease in the number of aircraft in flight during the night, dropping below 500 units. In contrast, the number peaks around 2 PM, reaching almost 3,000. Given that the trends over the two days are similar, they can be considered interchangeable. Therefore, we continue our estimations based on data from a single day, October 2, 2022. To ensure a realistic implementation of Swarm capable of handling stressful situations, we consider the scenario with the highest transaction volume: the peak air traffic hour.

Given that the ADS-B protocol has a communication range of 100 nmi (approximately 185 km), a single sender could potentially interact with a great number of target nodes, leading to a huge number of transactions. Such a volume would be redundant for the system's purposes and excessively computationally expensive. Therefore, in Swarm, when generating an ADS-B snapshot, the target node only creates a transaction if its distance from the sender falls within a predefined generation range.

We select a transaction generation range of 50 nmi, considered optimal, ensuring an average of nearly 19 target nodes per sender node. This range is broad enough to allow accurate validation of transactions through the heuristics explained in Section 4.3.1 while avoiding excessive system redundancy and the need for extreme computational resources from the underlying distributed ledger.

5.3.3 Block-Time. The *block-time* represents the interval between the generation of two consecutive blocks. The block-time should be configured to strike a tradeoff between performance and communication overhead. Lowering the block-time increases the frequency of block generation, which results in more information being stored for each airplane in the same amount of time. However, this also leads to increased communication overhead. A shorter block-time allows the

distributed ledger to remain nearly up-to-date in real-time. In practice, a very short block-time might reduce the likelihood that information from various aerovehicles can be included within the same block. This is due to the increased delays and complexities inherent in airborne networks, where topology changes frequently and connection stability varies. Moreover, setting the block-time too low may conflict with the time required to propagate each block after its creation—an operation that can take a non-negligible amount of time in real-world scenarios. On the other hand, assuming all other system configurations remain constant, a shorter block-time reduces the duration of transitory phases, such as during reputation initialization. This may also increase the penalty in reputation loss when misbehavior is detected by the heuristics embedded in Swarm. Conversely, a higher block-time would have the opposite effects—reducing communication overhead and increasing the probability of aggregating diverse information into a single block. However, this would also require a larger block size or result in a higher discard rate due to the block-size limit being reached. For these reasons, although the Solana blockchain adopted in Swarm supports a significantly high tps, enabling relatively low block-times (e.g., 0.4 seconds on average), such a low value is not suitable for our deployment scenario.

To further optimize performance, a Swarm implementation could consider a dynamic block-time mechanism that adjusts within a predefined range based on real-time network conditions. By monitoring factors such as propagation delay, network congestion, and node availability, the system could slightly increase or decrease the block-time to improve efficiency while maintaining overall system stability. However, such adjustments must be constrained to prevent adverse effects on other blockchain parameters.

In our deployment scenario, we empirically determined a block-time of 5 seconds to balance transaction finality, network stability, and block propagation delay. Setting a fixed block-time also provides a deterministic constraint, ensuring that other system parameters—such as block size—remain within expected operational limits, thus simplifying simulation environments. Nevertheless, the results and insights presented remain generalizable to scenarios involving a dynamically adjusted block-time, provided that suitable constraints are imposed to maintain system stability. Future iterations of Swarm may integrate adaptive mechanisms to fine-tune block production dynamically, further enhancing resilience under challenging network conditions.

5.3.4 Block-Size. While the system can discard transactions that exceed the block-size after they have been accepted, this should be avoided, as it may lead to the rejection of valid subsequent transactions that rely on discarded ones, rendering them illegitimate in the ledger. The estimations used to determine the A_t and R_t parameters lead to a maximum of $1,260 \cdot 19 \approx 24,000$ transactions generated per second. With a block-time fixed at 5 seconds, there may be a maximum of $24,000 \cdot 5 = 120,000$ transactions stored in each block. Consequently, setting the maximum block-size value to 120,000 transactions appears to be a suitable and reasonable choice.

5.4 Data Latency

In our approach, we identify four main components contributing to the latency between the time a target node receives an ADS-B message and the time when the same or a different node receives the distributed ledger block containing that message. These components are: (1) the time to generate an ADS-B snapshot, i.e., the transaction in our system that represents a summary of messages received during a specific timestamp through the ADS-B protocol; (2) the transmission time required to share the generated transaction with other nodes in the system; (3) the time to generate a block that includes the transaction; and (4) the time to share the newly generated block of the distributed ledger with the node that will use the information contained in it. The time required to generate a snapshot is deterministic and depends on the chosen timestamp unit. Similarly, the time that passes between the moment a transaction is received by the aggregator and when this transaction

is included in a new block, ready to be shared with other participants, is also deterministic. With a block time of 5 seconds, this delay is, in the worst case, equal to 5 seconds, with a range between 0 and 5 seconds. The other two components, however, cannot be precisely estimated due to network dynamics and varying node connectivity. Nevertheless, we can broadly estimate the time required for these components to be on the order of a few seconds.

It is important to note that our settings and framework are not intended for use in emergency situations. Instead, our framework's safety mechanisms rely on continuous tracking over several minutes to monitor potential anomalous situations. This ensures that even with blockchain-induced latencies, the system remains operationally effective, as decisions are made based on persistent and cumulative state awareness rather than single, short-term data points.

5.4.1 Delay-Induced Desynchronization. In the context of Swarm, *desynchronization* refers to a temporary state in which honest nodes have inconsistent views of the current set of valid transactions due to communication delays. This occurs when an ADS-B message or transaction arrives too late to be included in the current block and is instead processed in the subsequent one. As a result, nodes that did receive the message in time may accept and validate it, while others may temporarily reject it as invalid (e.g., due to missing context or inconsistent heuristics). Such desync events are not permanent and do not cause forks or finality violations, but they may lead to temporary disagreement on transaction validity or inconsistent reputation updates. Swarm mitigates this effect through progressive reputation updates. Once the delayed data is received and included in subsequent blocks, the views across nodes reconverge, restoring consistency and allowing honest participants to regain reputation over time.

To provide a concrete explanation of how message delay affects the probability of node desynchronization, we consider a timing-based model tied directly to the block-time. Let the block-time be $T_b = 5$ seconds. Assume that messages are broadcast uniformly at random within each block interval. If a message experiences a fixed delay Y , then it will only arrive within the same block if it was sent before time $T_b - Y$. Otherwise, it will be received in the next block, leading to a temporary inconsistency in node views and transaction acceptance. Under this assumption, the probability of desynchronization is given by

$$\Pr(\text{desync}) = \begin{cases} \frac{Y}{\text{block-time}}, & \text{if } Y < T_b \\ 1, & \text{if } Y \geq T_b \end{cases}$$

Figure 5 illustrates this relationship. For example, a delay of 2.5 seconds results in a 50% chance that a message arrives too late to be processed in the current block. While this model simplifies certain aspects of message propagation, it provides a reliable baseline for reasoning about desync under partial synchrony.

Figure 6 illustrates how a communication delay of approximately 15 seconds between two aircraft can lead to temporary inconsistencies in transaction validation. In this scenario, messages exchanged between Aircraft A and Aircraft B arrive too late to be included in the first three blocks, causing incomplete or unverifiable snapshots and, consequently, a possible reduction in reputation for one or both nodes. However, as shown in the diagram, starting from Block 4—after the delay window elapses—the delayed messages begin to arrive within the block-time interval. This enables successful validation of transactions, and the involved nodes begin to regain reputation through continued correct behavior.

6 System Parameter Tuning

This section outlines the steps to tune the Swarm's parameters, establishing a reliable baseline for evaluating the system's effectiveness. We evaluate the performance of various parameter settings

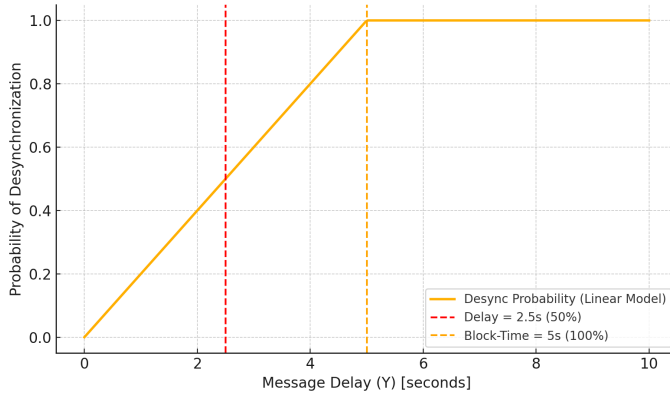


Fig. 5. Linear correlation between message delay Y and the probability of temporary desynchronization, assuming uniform broadcast timing.

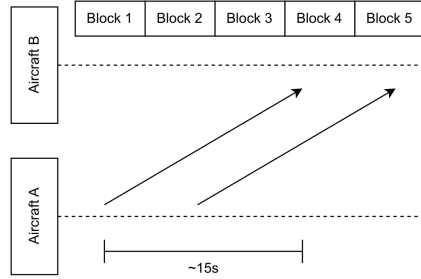


Fig. 6. Sequence diagram illustrating the impact of a 15-second message delay between two aircraft.

using two key metrics: the average reputation of participants (representative of their stake within the PoS consensus mechanism) and the number of accepted transactions. To do so, we rely on real-world air traffic data provided by the OpenSky Network [26, 37].

Due to the high computational demands of simulating a real-world scenario with the entire air traffic dataset, we reduce the data volume handled during simulations. This is achieved by selecting a subset of airports and considering only the ADS-B snapshots received from aircraft flying between these airports. Specifically, we select the 10 busiest European airports for the year 2022, which include Frankfurt (FRA), Germany; Heathrow (LHR), United Kingdom; Amsterdam Schiphol (AMS), Netherlands; Dublin (DUB), Ireland; Adolfo Suárez Madrid-Barajas (MAD), Spain; Charles de Gaulle (CDG), France; Leonardo da Vinci-Fiumicino (FCO), Italy; Vienna (VIE), Austria; Lisbon Portela (LIS), Portugal; and Athens (ATH), Greece.

6.0.1 Reputation Parameters. The system dynamically manages the reputation of network nodes, which reflects their stake in the PoS consensus mechanism. We carefully define the initial reputation values for both aircraft and airports. Additionally, we optimize the parameters that update node reputation over time.

System Initialization. As explained in Section 4.2, during the creation of the distributed ledger's genesis block, the system must select a set of airports as trusted entities. This step is essential to establish a reliable and verifiable foundation for the distributed ledger. These airports act as the first validators, ensuring that the initial state of the ledger is free from malicious or erroneous

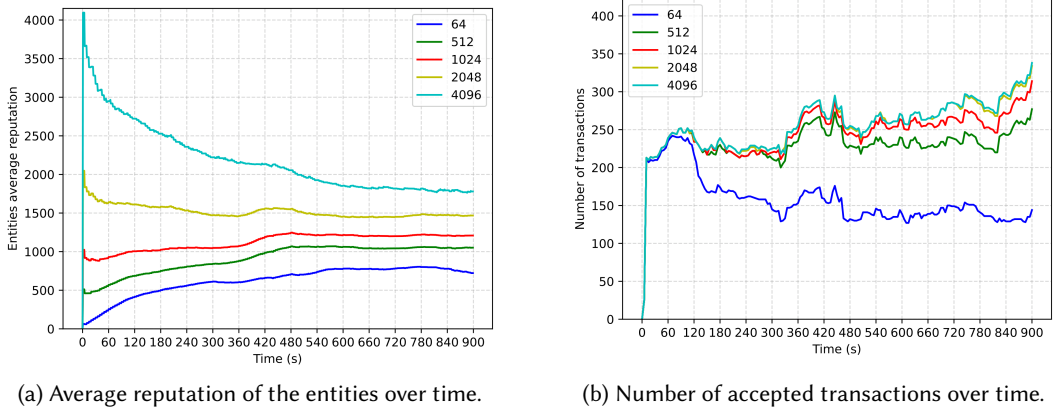
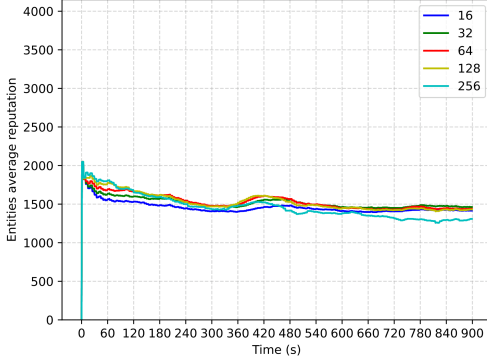


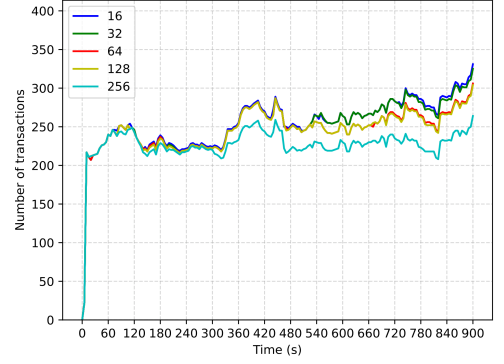
Fig. 7. Impact of different choices for the initial reputation value $\mathcal{R}ep$ on the system's performance.

entries before the network becomes fully operational. However, to maintain decentralization and prevent centralization risks, this trusted status is temporary. Once the system is initialized, these airports lose their privileged role, and all entities—including both aircraft and airports—operate under the same reputation-based validation process. This approach ensures that no single entity retains permanent authority, reducing the risk of collusion or control concentration while still providing a secure and structured initialization for the Swarm framework. For this reason, we initialize both aircraft and airports with the same reputation value $\mathcal{R}ep$. We test various values ranging from very low, such as 64, to very high, up to 4096, representing the maximum reputation value for each entity. As shown in Figure 7, the average reputation score of the entities converges over time for any initial value, proportionally to the chosen initial value. Furthermore, while all other values exhibit worse performance, setting the initial reputation at 2048 and 4096 guarantees the same level of performance in terms of accepted transactions within the system. Ultimately, we choose an initial reputation value of 2048. This ensures the proper functioning of the framework, even in case of attacks. Indeed, initializing the reputation of attackers with an excessively high value could give them an advantage, while starting honest entities with too low could significantly limit their ability to increase their reputation.

Reputation Update. Upon the generation of a block, the system updates the reputation of active participants based on the validity of the transactions in which they are involved, as outlined in Section 4.3.2. We define R_{Val} as the reputation increase factor and P_{Val} as the reputation decrease factor. To effectively manage nodes that may initially behave well but later start attacks, it is crucial to impose more significant penalties for malicious behavior than rewards for good behavior. Therefore, for all possible choices of R_{Val} , we set P_{Val} at twice the value of R_{Val} . Figure 8 illustrates the system performance across different factor values. Higher factor values lead to poorer performance because sporadic incoherent behaviors are heavily penalized. We select a value of R_{Val} equal to 32 and, consequently, a value of P_{Val} equal to 64, as a tradeoff between performance and the capacity of a node to increase its reputation over time. Moreover, to give more weight to recent transactions, the reputation of nodes not involved in any transactions for a certain number of blocks is gradually reduced. By artificially introducing some inactive nodes during simulations, we determined the optimal values for the parameters used by the system to update the reputation of such nodes, namely $\mathcal{I}N_{Blocks}$ and $\mathcal{I}N_{Factor}$. $\mathcal{I}N_{Blocks}$ specifies the time period after which the reputation of inactive nodes begins to decrease, while $\mathcal{I}N_{Factor}$ defines the rate of decrease. We tested values for $\mathcal{I}N_{Blocks}$



(a) Average reputation of the entities over time.



(b) Number of accepted transactions over time.

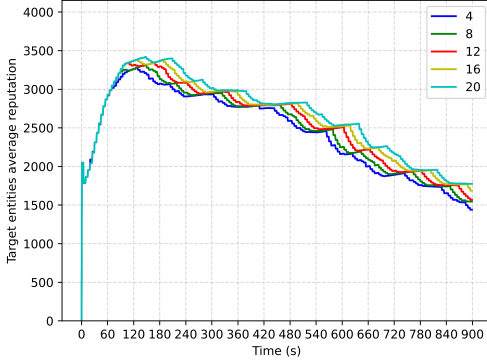
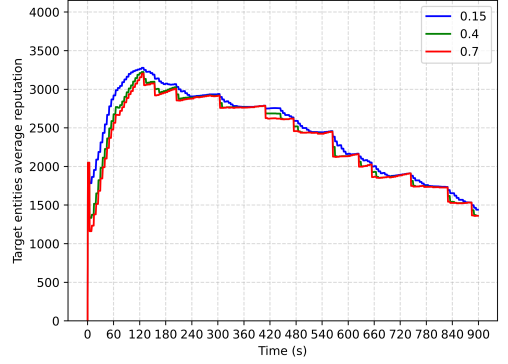
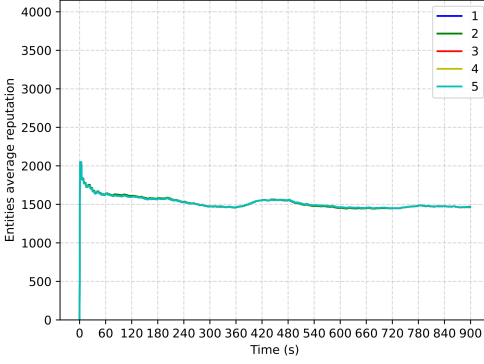
Fig. 8. Impact of different choices for the value R_{Val} on the system's performance.(a) Average reputation for different choices for the IN_{Blocks} parameter.(b) Average reputation for different choices for the IN_{Factor} parameter.

Fig. 9. Average reputation of artificially inactive nodes over time.

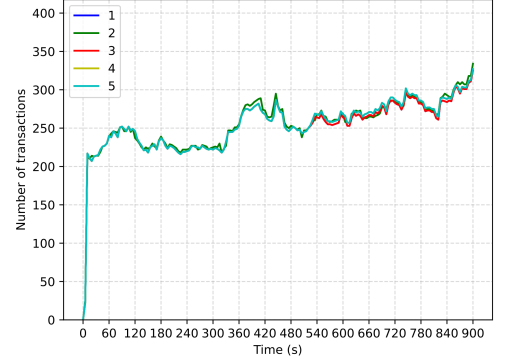
ranging from 4 to 20 blocks. As shown in Figure 9(a), the average reputation tends to converge to the same value across these settings. We selected a value of 4 blocks as a reasonable tradeoff, as it aligns well with the system's objectives. Figure 9(b) presents tests with different values of IN_{Factor} . We chose 0.15 as a reasonable value to maintain a gradual but steady decrease in reputation, favoring nodes that have been active in more recent time intervals over those that have not been actively participating in the system recently.

6.0.2 Heuristics Parameters. Parameters used in the consensus algorithm heuristics are evaluated and tuned. The system's robustness heavily depends on them, as an incorrect parameter selection can lead to potential malfunctions or harmful behaviors.

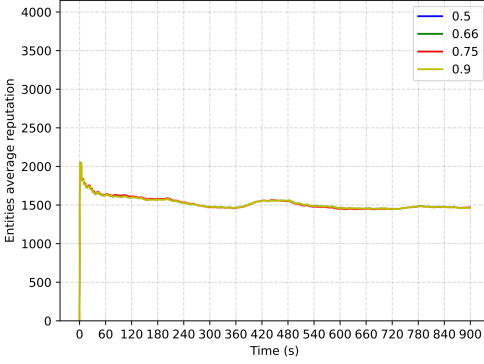
Location Validation. This heuristic checks the physical compliance of the aircraft locations stored in a transaction against the ones stored in the most recent blocks. We tune two parameters for this heuristic: LV_{Blocks} , the number of last generated blocks to check, and $LV_{Threshold}$, the minimum weighted percentage of locations stored in the previous blocks that must be consistent with the locations in the new transaction to confirm its legitimacy. The weight is proportional to the reputation of the entity that generated each previously stored transaction. We evaluate LV_{Blocks}



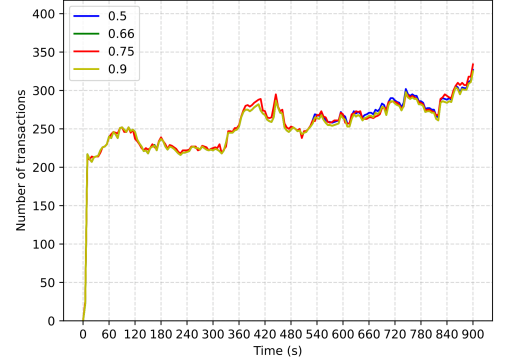
(a) Average reputation of the entities over time.



(b) Number of accepted transactions over time.

Fig. 10. Impact of different LV_{Blocks} parameter values on the system's performance.

(a) Average reputation of the entities over time.



(b) Number of accepted transactions over time.

Fig. 11. Impact of different $LV_{Threshold}$ parameter values on the system's performance.

values ranging from 1 to 5. As shown in Figure 10, the different values generally lead to similar results, both in terms of entities' average reputation and accepted transactions. This can be attributed to the significant redundancy of information within the system. Therefore, there is no need to evaluate a high number of recently generated blocks, as each block likely keeps track of most of the required information. This allows for keeping the computational overhead low. We set LV_{Blocks} to 2 as an effective tradeoff between the required computational and temporal complexity and the amount of information used to approve a new transaction. We observe similar results for the $LV_{Threshold}$ parameter, as shown in Figure 11. While a high threshold might also appear adequate based on these results, we choose 0.75 for a better tradeoff. This value considers potential real-world complications, such as technical issues that could lead to missing information and the rejection of legitimate transactions.

Ghost Aircraft Penalization. This heuristic aims to evaluate the presence of an aircraft in the declared location through collaboration with nearby entities. We tune the $GAP_{Threshold}$ parameter, which is the threshold value a new transaction must exceed to be considered valid by this heuristic. Unlike the *Location Validation* heuristic, Figure 12 shows less homogeneous trends. This suggests the impact of $GAP_{Threshold}$ on the *Ghost Aircraft Penalization* heuristic. We choose the value 0.66 to

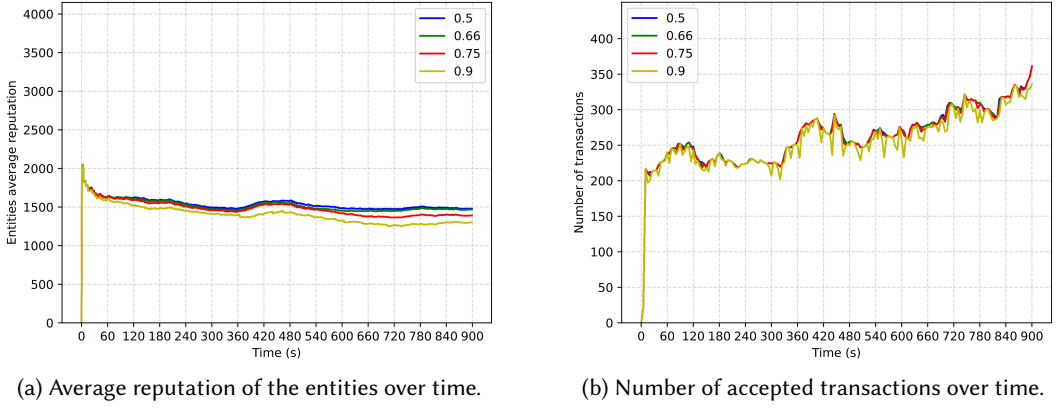


Fig. 12. Impact of different $GAP_{Threshold}$ parameter values on the system's performance.

both ensure that no invalid transaction is accepted and to address potential attacks attempting to invalidate the presence of a legitimate entity.

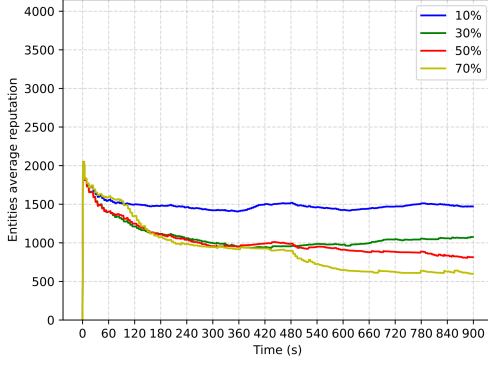
7 Experimental Evaluation under Attack Scenarios

As mentioned in Section 3, ATC might potentially be subject to various attacks from different actors. By sending malicious ADS-B communications, attackers might simulate the presence of nonexistent aircraft in the airspace and disrupt traffic control services. Distributed ledger technology provides a decentralized and immutable solution for sharing air traffic information among entities and establishing the reputation of participants, potentially helping to identify malicious attempts over the ADS-B protocol, including flooding, ghost injection, and message tampering. However, incorrect parameter configurations might render the distributed ledger unable to properly deal with attackers. Moreover, motivated attackers might also perform more sophisticated attacks that first compromise the distributed ledger and then disrupt traffic control services. For these reasons, we test Swarm under different conditions and attack scenarios to assess its capability to deal with ADS-B-based attacks and more sophisticated attacks targeting both ADS-B and the distributed ledger.

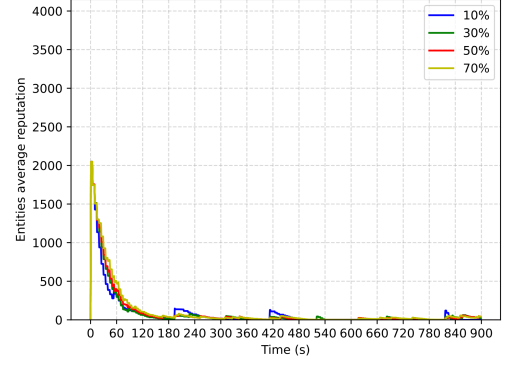
7.1 Attack Simulation Setup

We employ real-world air traffic data from the OpenSky Network [26, 37]. To reduce computational demands in the simulations while maintaining real-world accuracy, we focus on aircraft flying between the 10 busiest European airports in 2022. Different attack scenarios are simulated by configuring a portion of the nodes to behave maliciously and damage the system.

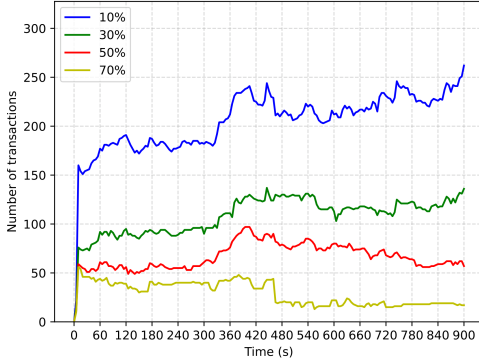
Figure 13 shows the behavior of Swarm by varying the percentage of malicious nodes. Intuitively, as the percentage of malicious participants increases, the number of rejected transactions also rises. Furthermore, as the number of malicious entities grows, the average reputation of honest entities decreases due to the increasing control that the attackers gain over the system. It is important to notice how the assumption of up to 70% malicious nodes in experiments is not representative of the security threshold expected under real-world consensus conditions. Rather, it serves as a stress test hypothesis to evaluate the robustness of the proposed heuristics under extreme adversarial behavior. In these simulated scenarios, malicious nodes attempt to inject or manipulate data but do not control consensus or collude to subvert block validation. The consensus process itself remains intact, assuming the standard security bounds of a reputation-based PoS system. In practical deployments, a sustained presence of greater than 33% malicious nodes could compromise consensus integrity. Our choice to simulate up to 70% malicious data sources is strictly



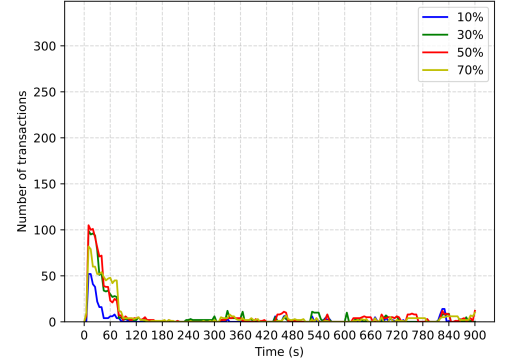
(a) Average reputation of honest entities over time.



(b) Average reputation of malicious entities over time.



(c) Number of accepted transactions over time.



(d) Number of rejected transactions over time.

Fig. 13. Evaluation of the different choices for the percentage of malicious nodes within the system.

for demonstration purposes to showcase Swarm's ability to detect and isolate malicious inputs even when the majority of data sources are adversarial, without implying that the consensus protocol itself would remain secure under those conditions.

For the following tests, we select a 10% level of malicious participants in the system, as it provides a good balance between realism and the necessity to test a suitable worst-case scenario. By demonstrating resilience in such a scenario, our framework ensures robustness against various types of attacks that could threaten its proper functioning and cause significant harm to the provided service. To reduce noise in the system and decrease the computational requirements of the simulation, the framework halts the generation of transactions from entities that have reached a reputation score of 0. Moreover, to maintain a homogeneous 10% level of active malicious participants, the framework periodically introduces new malicious entities into the system.

7.2 Results Under Attack Scenarios

In this section, we present the results of the experimental evaluation conducted on Swarm against attacks directed to compromise various aspects of both the ADS-B communications and the distributed ledger. We simulate attacks acting at the transaction, block, and consensus levels. At the transaction level, we simulate an attacker generating and broadcasting fake transactions or flooding the system with spam transactions. At the block level, we simulate attempts to modify blocks, compromising the validation algorithm and altering transaction validity tags, reputation scores, or

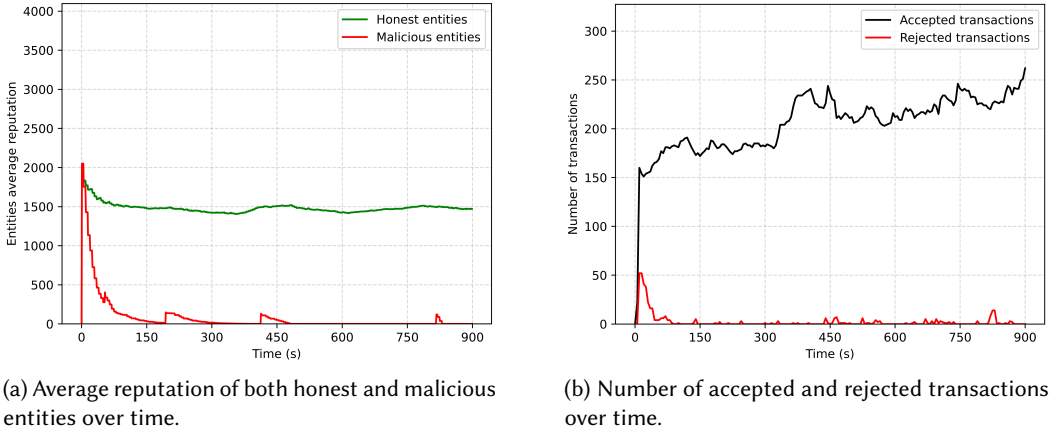


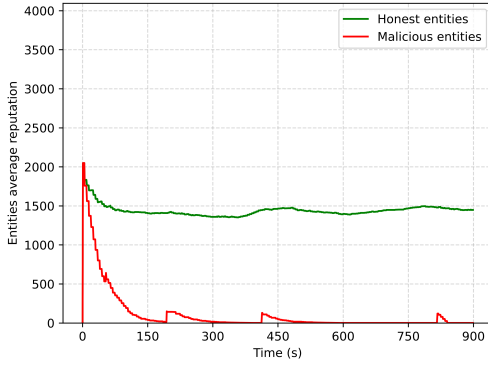
Fig. 14. Impact of the *Illegal Transaction Generation* attack on the system.

activity flags. At the consensus level, we simulate attempts by an attacker to gain a majority of the consensus power to reject legitimate blocks or force the system into accepting forged ones. The system's resilience is measured according to the evolution of the average reputation of honest and malicious participants over time. Specifically, the system can be considered resilient to a specific attack if the average reputation of honest participants stays above a threshold necessary for proper system functioning, while the reputation of malicious entities drops to zero due to the penalties imposed by the heuristics specifically designed to mitigate harmful behaviors.

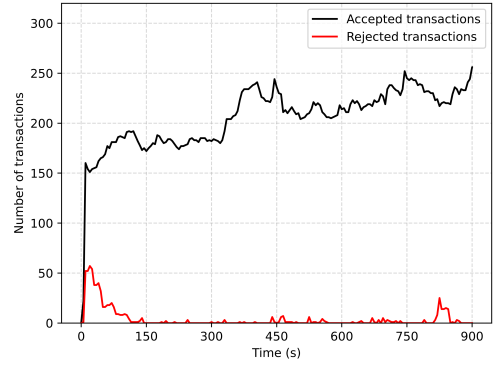
Illegal Transaction Generation Attack. In the *Illegal Transaction Generation* attack, each malicious participant generates a high number of transactions with identical sender and receiver IDs to artificially boost its own reputation. The attacker's goal is to achieve a high reputation score, enabling easier participation in the key consensus steps, with the aim of compromising the system at the consensus level. This attack can be prevented by the system's *Illegal Transaction Filtering* heuristic, described in Section 4.3.1, which identifies and discards transactions with identical sender and receiver IDs. Figure 14 illustrates the behavior of Swarm under an *Illegal Transaction Generation* attack, demonstrating the system's resilience to such threats.

Random Transaction Generation Attack. In the *Random Transaction Generation* attack, the attacker forges a substantial number of transactions, each with different and randomly generated data, to add noise to the system and fill up the blocks. The attacker's goal is to threaten the system at the transaction level by disrupting its proper functioning through the flooding of meaningless data, carrying out a DoS attack. Swarm implements two heuristics against this type of threat: *Location Validation* and *Ghost Aircraft Penalization*, detailed in Section 4.3.1. Both the heuristics label a transaction generated by the *Random Transaction Generation* attack as invalid since it is highly unlikely that the randomly generated sender location is physically consistent with those previously stored in the system and that it can be confirmed by the entities actually nearby this location at the claimed time instant. The Swarm's good capability to deal with the *Random Transaction Generation* attack is confirmed by our tests, as shown in Figure 15.

Teleport Attack. The *Teleport* attack is another type of malicious behavior threatening the system at the transaction level. Its goal is to generate transactions that are legitimate in every field except for the target's location, where the coordinates are randomly generated. As with the *Random Transaction Generation* attack, *Location Validation* and *Ghost Aircraft Penalization* heuristics are employed to protect Swarm. Since the attacker is generating a random location, the new position is

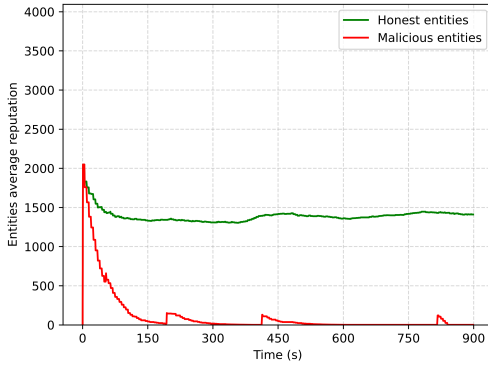


(a) Average reputation of both honest and malicious entities over time.

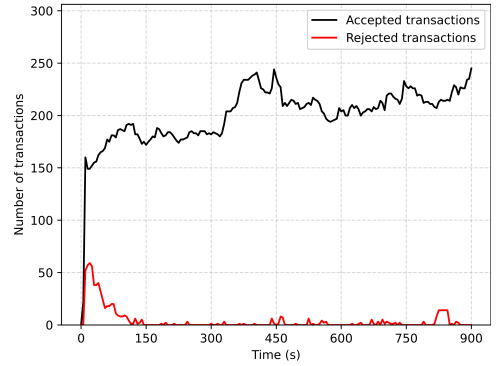


(b) Number of accepted and rejected transactions over time.

Fig. 15. Impact of the *Random Transaction Generation* attack on the system.



(a) Average reputation of both honest and malicious entities over time.



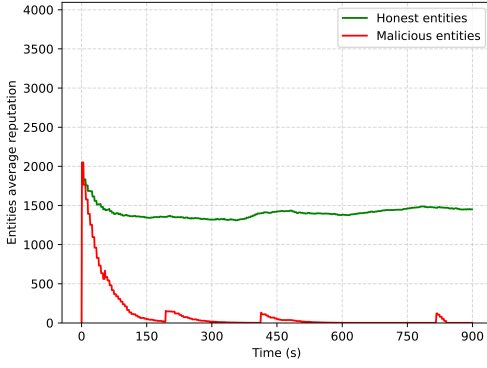
(b) Number of accepted and rejected transactions over time.

Fig. 16. Impact of the *Teleport Attack* on the system.

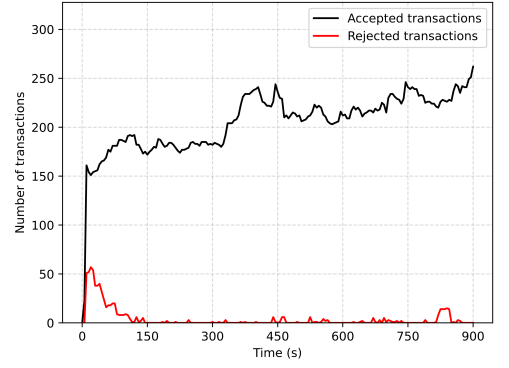
highly likely to result as physically non-compliant with the previous ones and will not be confirmed by a sufficient weighted percentage of nodes. As a result, Swarm demonstrates its capability to identify and block such types of attacks, as shown in Figure 16.

Fake Position Advertisement Attack. The *Fake Position Advertisement* attack, similar to the *Teleport* attack, is a transaction-level attack. However, unlike the *Teleport* attack which generates a completely new random target location for each transaction, the attacker in this scenario adds a fixed offset to the actual coordinates. For both latitude and longitude, the offset is established to be large enough to fall outside the expected range, disrupting the system's operation and deceiving it about the attacker's real location. Since the attacker adds a significant offset to their real location, the *Location Validation* and *Ghost Aircraft Penalization* heuristics effectively detect the *Fake Position Advertisement* attack. This maintains the integrity of transactions and the overall system's reliability, as shown in Figure 17.

Copycat Attack. The *Copycat* attack involves choosing a node and copying its transactions, replacing the target node's ID with the attacker's ID. The goal of the attacker is to mimic an honest node and appear in false positions within the airspace. This would create serious issues for legitimate

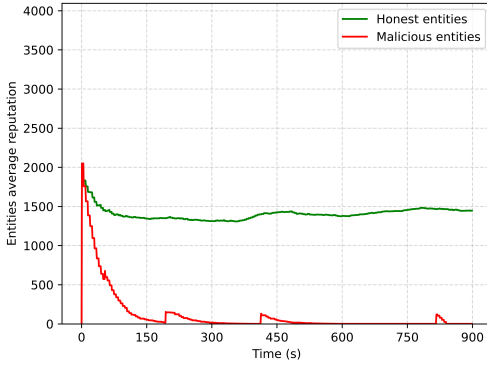


(a) Average reputation of both honest and malicious entities over time.

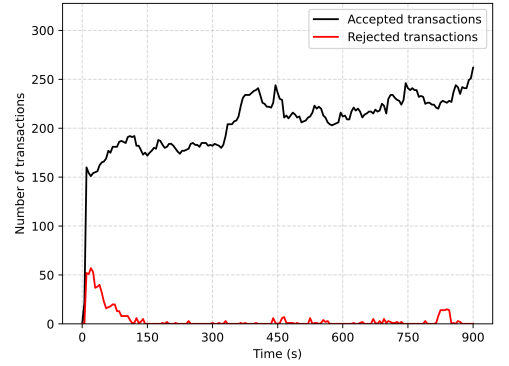


(b) Number of accepted and rejected transactions over time.

Fig. 17. Impact of the *Fake Position Advertisement* attack on the system.



(a) Average reputation of both honest and malicious entities over time.

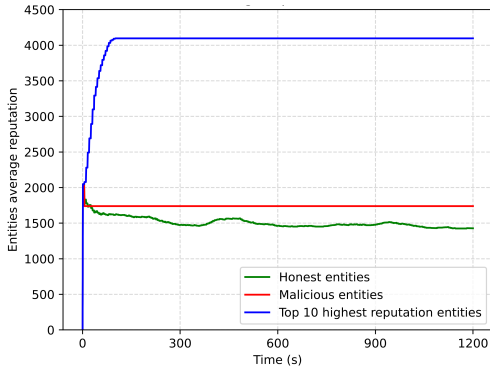


(b) Number of accepted and rejected transactions over time.

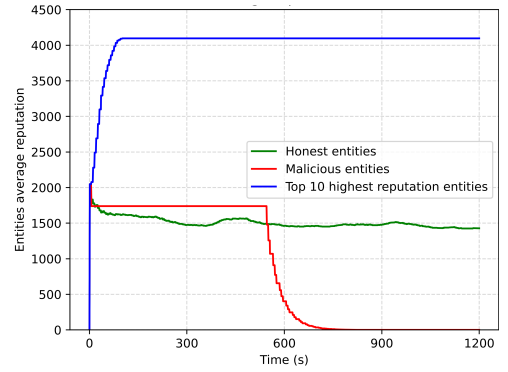
Fig. 18. Impact of the *Copycat Attack* on the system.

users operating near the false locations claimed by the attacker. Once again, the *Location Validation* heuristic helps identify malicious transactions generated by this attack. The new locations claimed by the attacker are unlikely to be physically consistent with the actual ones previously confirmed by the system. Moreover, the *Ghost Aircraft Penalization* mechanism significantly helps mitigate the effects of the *Copycat Attack*. Since the attacker is not actually broadcasting ADS-B messages from the false location spoofed by copying a legitimate transaction, the claimed position will not be confirmed by a sufficient weighted percentage of entities. The system behavior under this attack is shown in Figure 18.

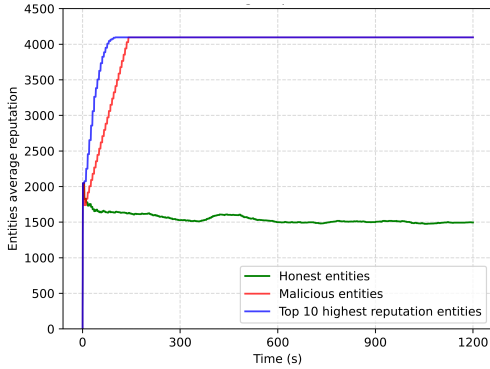
Multi-boost Attack. The *Multi-boost* attack poses a threat to the system at both transaction and consensus levels and requires a coordinated effort by multiple attackers. During the first phase of the attack, the malicious nodes are near each other and exchange ADS-B messages. Since this activity leads to the generation of valid transactions, it generally increases the reputation of the attackers. Once the nodes have legitimately achieved a high reputation score, one of them gradually starts generating transactions with false locations that are consistent with those generated by the other attackers. At the transaction level, as long as the malicious nodes declare to be located within



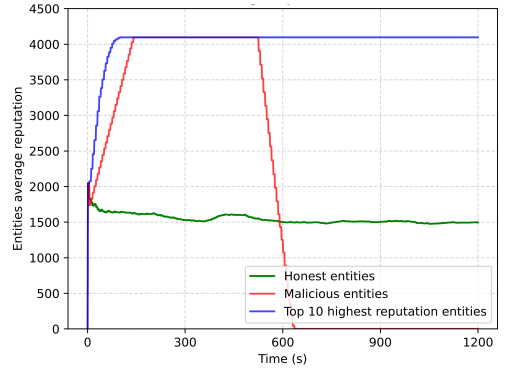
(a) Static *Multi-boost* attack scenario with a number of attackers less than N .



(b) Dynamic *Multi-boost* attack scenario with a number of attackers less than N .



(c) Static *Multi-boost* attack scenario with a number of attackers greater than N .



(d) Dynamic *Multi-boost* attack scenario with a number of attackers greater than N .

Fig. 19. Impact of the *Multi-boost* attack on the system, both for a static and a dynamic scenario, and for a number of attackers less and greater than N . Here, N represents the number of the most recent interacting entities with node i that influence its reputation update.

their reciprocal transaction generation range, the behavior is considered legitimate; therefore, no action is taken and no penalties are applied. However, at a certain point, the aircraft gradually sending false locations will reach a distance that exceeds the transaction generation range of Swarm. Since each transaction contains the claimed locations of both the sender and target nodes, the *Location Validation* heuristic will invalidate any transaction whose distance between the declared locations is greater than the transaction generation range. Moreover, the *Ghost Aircraft Penalization* heuristic will also penalize any declaration that is not compliant with the declaration of other nearby legitimate nodes.

At the consensus level, the *Multi-boost* attack is mitigated by the scaling factor used in the reputation update strategy. As presented in Section 4.3.2, this factor prevents the mutual reputation boost of malicious nodes. Specifically, the scaling factor ensures that the reputation update for node i depends on the most recent N entities that have interacted with node i . Consequently, a suitable choice for the parameter N guarantees that if the number of attackers is less than N , they cannot threaten the system. This parameter should be chosen under the assumption that the number of attackers is unlikely to exceed N . Figure 19 illustrates the evolution of the nodes' reputations when the number of attackers is fewer than N and greater than N . In both scenarios, the simulations

demonstrate the system's behavior in static and dynamic situations. In the static case, the attackers remain in the same positions without moving. In the dynamic scenario, coherently with the *Multi-Boost* attack strategy, one of them gradually starts generating transactions with false locations that are consistent with those generated by the other attackers. To more exhaustively demonstrate the overall behavior of Swarm, we have depicted in blue the reputation trends of the 10 nodes with the highest reputation over time. This visualization shows how, in realistic scenarios with fewer than N attackers, the reputation of malicious entities is hindered from growing compared to that of legitimate ones, thanks to the scaling factor. Therefore, they are significantly disadvantaged in participating in key consensus steps due to the presence of legitimate entities that achieve higher reputation scores through honest behavior. Moreover, by observing the dynamic situation plots, it's possible to identify the moment when the attacker's distance from other malicious entities prevents them from generating plausible transactions. From that point on, the reputation of the malicious entities decreases to zero, demonstrating the system's ability to manage this attack at the transaction level. As expected, if the number of attackers exceeds N , the system can be compromised at the consensus level. Nevertheless, Swarm will still penalize the attackers at the transaction level if any of them generate invalid transactions.

8 Formal Security Guarantees

In this section, we formally demonstrate the security properties of the Swarm framework. We present worst-case bounds under which Swarm remains robust even in the presence of Byzantine actors, and proves that under such conditions, the probability of an attacker fully controlling consensus decisions remains strictly less than 1.

To this end, we provide two formal demonstrations:

- (1) A proof that Swarm guarantees safety and liveness provided that faulty nodes hold less than one-third of the effective reputation.
- (2) A bound showing that the probability of adversarial leader control in any round is strictly less than one, ensuring unpredictability and fairness in leader election.

8.1 Safety and Liveness

We provide a formal argument demonstrating that the Swarm framework guarantees both *safety* and *liveness*, assuming worst-case adversarial behavior constrained by the system's reputation-based design.

Definitions and Assumptions. Let:

- $S(t)$ be the total sum of reputation scores of all nodes participating in consensus at time t .
- f be the total number of Byzantine (faulty) nodes.
- h be the number of honest nodes, with $n = f + h$ as the total number of participants.
- $R_{\max}$ be the maximum reputation any node can hold (as enforced by Swarm, e.g., 4096).
- $R_{\min}^{\text{decay}}$ be the minimum reputation maintained by an honest node under decay.
- $R_{\text{faulty}}(t) = \sum_{i \in \mathcal{F}} R_i(t) \leq f \cdot R_{\max}$ be the total reputation controlled by faulty nodes at time t .
- The network is partially synchronous: messages are eventually delivered within a bounded delay.

Adversarial Bound. To ensure robustness, we require that the total reputation of faulty nodes remains strictly less than one-third of the total system reputation. In the worst case, this becomes:

$$f \cdot R_{\max} < \frac{1}{3} \left(f \cdot R_{\max} + h \cdot R_{\min}^{\text{decay}} \right)$$

Solving this inequality for $R_{\min}^{\text{decay}}$ yields the condition:

$$R_{\min}^{\text{decay}} > \frac{2f \cdot R_{\max}}{h}$$

This defines the minimum reputation honest nodes must retain to ensure system security and progress, even under maximum adversarial stake.

Safety. In Swarm, a block is committed only if validated by a majority of reputation-weighted validators. If faulty nodes control less than $1/3$ of the effective reputation, any committed block must be approved by at least some honest validators. Since honest nodes apply validation heuristics (e.g., location consistency, ghost aircraft detection), they will not approve conflicting or invalid blocks. Thus, under the bound above, no two honest nodes can finalize conflicting blocks.

Liveness. Any valid transaction submitted by an honest node is propagated through the P2P network. With leader selection weighted by reputation, and honest nodes holding more than $2/3$ of the total reputation, an honest leader will eventually be selected. Once chosen, the leader includes the valid transaction in a block, which is then validated by the honest majority and added to the ledger. Hence, valid transactions are eventually committed.

Conclusion. Under the constraint:

$$R_{\min}^{\text{decay}} > \frac{2f \cdot R_{\max}}{h}$$

the Swarm framework guarantees:

- **Safety:** No two honest nodes will commit conflicting blocks.
- **Liveness:** Every valid transaction from an honest node will eventually be included and finalized.

This bound captures the worst-case scenario in which all faulty nodes have maximum reputation. Swarm's enforced reputation cap ($R_{\max} = 4096$) and decay mechanisms ensure that this condition is realistically sustainable. Consequently, the framework maintains consensus integrity and progress even in adversarial environments.

8.2 Leader Selection Probability Bound

Let $\Pi(t)$ be the probability that the leader selected at time t is a faulty node. In Swarm, leader election follows a reputation-weighted random selection, where the probability of a node being selected as leader is proportional to its reputation relative to the total reputation in the system.

Let:

- f be the number of faulty nodes.
- $R_{\max}$ be the maximum reputation of any node (as enforced by Swarm).
- $S(t)$ be the total reputation of all participating nodes at time t .
- $R_{\text{faulty}}(t) \leq f \cdot R_{\max}$ be the total reputation held by faulty nodes.

Then, by definition of the reputation-weighted leader selection:

$$\Pi(t) = \frac{R_{\text{faulty}}(t)}{S(t)} \leq \frac{f \cdot R_{\max}}{S(t)}$$

To ensure that no adversary can deterministically control leader selection, we require:

$$\Pi(t) < 1 \quad \Leftrightarrow \quad f \cdot R_{\max} < S(t)$$

This condition is strictly weaker than the earlier safety bound $f \cdot R_{\max} < \frac{1}{3}S(t)$, and is always satisfied when the safety condition holds. Therefore, under the standard Swarm configuration,

where the reputation of faulty nodes is bounded and the honest nodes maintain nonzero reputation, the attacker cannot guarantee control of the leader selection process:

$$\Rightarrow \Pi(t) < 1$$

Hence, leader selection remains probabilistic and decentralized, ensuring fairness and resistance to adversarial capture of the consensus process.

9 Limitations

While our experimental evaluation under attack scenarios demonstrates that Swarm provides a decentralized and resilient framework for enhancing the security of ADS-B communications, some limitations remain.

A key challenge in blockchain-based reputation systems is the potential linkage between reputation scores and entity identities. If reputation gains are not anonymous, malicious actors could exploit this identity-reputation association to manipulate trust dynamics, launch targeted attacks, or undermine fairness in the system. Attackers might attempt to compromise the reputation system by selectively targeting nodes with known high or low trust scores. Although none of the attack scenarios we tested were able to compromise the system, the possibility of new, sophisticated attacks with sudden consequences cannot be ruled out. To further mitigate these risks, we may consider integrating recent cryptographic techniques into Swarm, such as scored anonymous credentials [9] and zero-knowledge proofs [14]. These techniques would allow nodes to prove their reputation status without directly revealing their identity or absolute trust score, thereby reducing the risk of targeted attacks while maintaining system integrity.

Moreover, potential network instability and delays present challenges in transaction validation. For instance, if a transaction is received and processed just before the block formation deadline, it may not have sufficient time to be cross-verified by nearby nodes, leading to erroneous penalties or endorsements. Similarly, if network delays are particularly high, especially in regions with sparse node coverage, transactions might be incorrectly rejected or accepted due to delayed communication, preventing some transactions from reaching validators in time. This could lead to temporary inconsistencies in block formation, affecting the reputation of legitimate nodes. However, as the system operates over multiple block intervals, these inconsistencies are expected to self-correct over time as nodes regain the ability to share their transactions. Likewise, in areas with low node density, there may be insufficient cross-validation of aircraft positions, increasing the risk of erroneous transactions in the short term.

10 Conclusions

This work introduced Swarm, a decentralized distributed ledger-based data validation framework built on top of the ADS-B protocol, aimed at enhancing ATC security. By leveraging the cooperation of distributed nodes, Swarm collects real-time air traffic information in the form of transactions, which are stored in the distributed ledger after rigorous validity checks. These checks are performed through a comprehensive set of heuristics tailored to real-world scenarios. Swarm can be integrated into existing ATC infrastructure without requiring any modifications to the ADS-B protocol. We have evaluated the feasibility, system configurations, and optimal parameter values of Swarm under realistic scenarios using real-world air traffic data. Additionally, we have assessed the robustness of Swarm under exhaustive attack scenarios, demonstrating its capability to enhance the security of the aviation domain. Future research should involve evaluating additional consensus algorithms to determine their impact on system performance. This would include an analysis of both computational efficiency and time constraints, with a particular focus on how these factors interact with hardware limitations in the aviation sector.

References

- [1] International Energy Agency. 2020. World Air Passenger Traffic Evolution (1980-2020). Retrieved from <https://www.iaea.org/data-and-statistics/charts/world-air-passenger-traffic-evolution-1980-2020>. Accessed: 2023-05-30.
- [2] Muhammad Salek Ali, Koustabh Dolui, and Fabio Antonelli. 2017. IoT data privacy via blockchains and IPFS. In *Proceedings of the Seventh International Conference on the Internet of Things*. 1–7.
- [3] Azza Allouch, Omar Cheikhrouhou, Anis Koubâa, Khalifa Toumi, Mohamed Khalgui, and Tuan Nguyen Gia. 2021. Utm-chain: Blockchain-based secure unmanned traffic management for internet of drones. *Sensors* 21, 9 (2021), 3049.
- [4] Tomaso Aste, Paolo Tasca, and Tiziana Di Matteo. 2017. Blockchain technologies: The foreseeable impact on society and industry. *Computer* 50, 9 (2017), 18–28.
- [5] Leemon Baird. 2016. The swirlds hashgraph consensus algorithm: Fair, fast, byzantine fault tolerance. *Swirlds Tech Reports SWIRLDS-TR-2016-01, Tech. Rep* 34 (2016), 9–11.
- [6] Frederico Baptista, Marina Dehez-Clementi, and Jonathan Detchart. 2024. DFly: A publicly auditable and privacy-preserving UAS traffic management system on blockchain. *Drones* 8, 8 (2024). DOI:<https://doi.org/10.3390/drones8080410>
- [7] Omar Cheikhrouhou and Anis Koubâa. 2019. Blockloc: Secure localization in the internet of things using blockchain. In *2019 15th International Wireless Communications & Mobile Computing Conference (IWCMC)*. IEEE, 629–634.
- [8] Sherman SM Chow. 2007. Running on karma—P2P reputation and currency systems. In *Cryptology and Network Security: 6th International Conference, CANS 2007, Singapore, December 8-10, 2007. Proceedings* 6. Springer, 146–158.
- [9] Sherman SM Chow, Jack PK Ma, and Tsz Hon Yuen. 2023. Scored anonymous credentials. In *International Conference on Applied Cryptography and Network Security*. Springer, 484–515.
- [10] International civil aviation organization Asia and Pacific office. [n. d.]. ADS-B Implementation and operations guidance document. Retrieved from <https://www.icao.int/APAC/Documents/edocs/cns/AIGD%20Edition%202015.0.pdf>
- [11] Marina Dehez Clementi, Nicolas Larrieu, Emmanuel Lochin, Mohamed Ali Kaafar, and Hassan Asghar. 2019. When air traffic management meets blockchain technology: A blockchain-based concept for securing the sharing of flight data. In *2019 IEEE/AIAA 38th Digital Avionics Systems Conference (DASC)*. Ieee, 1–10.
- [12] Andrei Costin, Aurélien Francillon, et al. 2012. Ghost in the air (traffic): On insecurity of ADS-B protocol and practical attacks on ADS-B devices. *Black Hat USA* 1 (2012), 1–12.
- [13] Nabil El Ioini and Claus Pahl. 2018. A review of distributed ledger technologies. In *On the Move to Meaningful Internet Systems. OTM 2018 Conferences*. Hervé Panetto, Christophe Debruyne, Henderik A. Proper, Claudio Agostino Ardagna, Dumitru Roman, and Robert Meersman (Eds.). Springer International Publishing, Cham, 277–288.
- [14] Uriel Fiege, Amos Fiat, and Adi Shamir. 1987. Zero knowledge proofs of identity. In *Proceedings of the Nineteenth Annual ACM Symposium on Theory of Computing*. 210–217.
- [15] Farah Hasin, Tahmina Haque Munia, Nazmun Nahar Zumu, and Kazi Abu Taher. 2021. ADS-B based air traffic management system using ethereum blockchain technology. In *2021 International Conference on Information and Communication Technology for Sustainable Development (ICICT4SD)*. IEEE, 346–350.
- [16] ICAO. 2018. *Annex 11 to the Convention on International Civil Aviation—Air Traffic Services*. Technical Report. ICAO.
- [17] IOST. 2017. IOST: The Next-Generation, Secure, Highly Scalable Ecosystem for Online Services. Retrieved from <https://iost.io/624/>
- [18] Thabet Kacem, Duminda Wijesekera, and Paulo Costa. 2018. ADS-Bsec: A holistic framework to secure ADS-B. *IEEE Transactions on Intelligent Vehicles* 3, 4 (2018), 511–521.
- [19] Eleftherios Kokoris-Kogias, Philipp Jovanovic, Linus Gasser, Nicolas Gailly, Ewa Syta, and Bryan Ford. 2018. Omniledger: A secure, scale-out, decentralized ledger via sharding. In *2018 IEEE Symposium on Security and Privacy (SP)*. IEEE, 583–598.
- [20] Mahtab Kouhizadeh and Joseph Sarkis. 2018. Blockchain practices, potentials, and perspectives in greening supply chains. *Sustainability* 10, 10 (2018), 3652.
- [21] Xin Lu, Zhijun Wu, Yanhan Wu, Qing Wang, and Yue Yin. 2021. ATMChain: Blockchain-based solution to security problems in air traffic management. In *2021 IEEE/AIAA 40th Digital Avionics Systems Conference (DASC)*. IEEE, 1–8.
- [22] Davide Maffiola, Stefano Longari, Michele Carminati, Mara Tanelli, and Stefano Zanero. 2022. GOLIATH: A decentralized framework for data collection in intelligent transportation systems. *IEEE Transactions on Intelligent Transportation Systems* 23, 8 (2022), 13372–13385. DOI:<https://doi.org/10.1109/TITS.2021.3123824>
- [23] Mohsen Riahi Manesh and Naima Kaabouch. 2017. Analysis of vulnerabilities, attacks, countermeasures and overall risk of the automatic dependent surveillance-broadcast (ADS-B) system. *International Journal of Critical Infrastructure Protection* 19 (2017), 16–31. DOI:<https://doi.org/10.1016/j.ijcip.2017.10.002>
- [24] Donald McCallie, Jonathan Butts, and Robert Mills. 2011. Security analysis of the ADS-B implementation in the next generation air transportation system. *International Journal of Critical Infrastructure Protection* 4, 2 (2011), 78–87.
- [25] Satoshi Nakamoto. 2008. Bitcoin: A Peer-to-Peer Electronic Cash System. White paper.
- [26] OpenSky Network. 2023. OpenSky Network. Retrieved from <https://opensky-network.org/>

- [27] Airports Authority of India. [n. d.]. Automatic Dependent Surveillance—Broadcast (ADS-B) Out Based Ats Surveillance Services. Retrieved from https://web.archive.org/web/20160304051131/http://www.aai.aero/misc/AIPS_2014_18.pdf
- [28] Wei-Jun Pan, Zi-Liang Feng, and Yang Wang. 2012. ADS-B data authentication based on ECC and X. 509 certificate. *Journal of Electronic Science and Technology* 10, 1 (2012), 51–55.
- [29] Swathi Pennapareddy and K. Natarajan. 2022. Securing ADS-B data transmissions using blockchain: a comprehensive survey and analysis. *Aircraft Engineering and Aerospace Technology* 95, 3 (October 2022), 452–463. DOI: <https://doi.org/10.1108/AEAT-02-2022-0058>
- [30] Serguei Yu. Popov. 2015. The Tangle. Retrieved from <https://api.semanticscholar.org/CorpusID:4958428>
- [31] Ronald J Reisman. 2019. Air traffic management blockchain infrastructure for security, authentication, and privacy. In *AIAA Scitech Forum*. NASA Technical Report ARC-E-DAA-TN63825.
- [32] William R. Richards, Kathleen O'Brien, and Dean C. Miller. 2010. New air traffic surveillance technology. *Boeing Aeromagazine* 2, 7 (2010).
- [33] Roberto Saia, Alessandro Sebastian Podda, Livio Pompianu, Diego Reforgiato Recupero, and Gianni Fenu. 2021. A blockchain-based distributed paradigm to secure localization services. *Sensors* 21, 20 (2021), 6814.
- [34] Fahad Saleh. 2021. Blockchain without waste: Proof-of-stake. *The Review of Financial Studies* 34, 3 (2021), 1156–1190.
- [35] Nishant Sapra, Inlak Shaikh, and Ashutosh Dash. 2023. Impact of proof of work (PoW)-based blockchain applications on the environment: A systematic review and research agenda. *Journal of Risk and Financial Management* 16, 4 (2023), 218.
- [36] Dinesh Sathyamoorthy. 2013. Global navigation satellite system (GNSS) spoofing: A review of growing risks and mitigation steps. In *Defence S&T Technical Bulletin*, Vol. 6. Science & Technology Research Institute for Defence (STRIDE), 42–61.
- [37] Matthias Schäfer, Martin Strohmeier, Vincent Lenders, Ivan Martinovic, and Matthias Wilhelm. 2014. Bringing up OpenSky: A large-scale ADS-B sensor network for research. In *IPSN'14, Proceedings of the 13th International Symposium on Information Processing in Sensor Networks (part of CPS Week), April 15-17, 2014, Berlin, Germany*, Adam Wolisz, Jie Liu, and Lin Zhong (Eds.). IEEE/ACM, 83–94. Retrieved from <http://dl.acm.org/citation.cfm?id=2602350>
- [38] Christophe Schinckus. 2021. Proof-of-work based blockchain technology and anthropocene: An undermined situation? *Renewable and Sustainable Energy Reviews* 152 (2021), 111682. DOI: <https://doi.org/10.1016/j.rser.2021.111682>
- [39] Skybrary. [n. d.]. Automatic Dependent Surveillance—Broadcast (ADS-B). Retrieved from <https://skybrary.aero/articles/automatic-dependent-surveillance-broadcast-ads-b>. Accessed: 2024-07-01.
- [40] Martin Strohmeier, Vincent Lenders, and Ivan Martinovic. 2013. Security of ADS-B: State of the art and beyond. arXiv:1307.3664. Retrieved from <https://arxiv.org/abs/1307.3664>
- [41] Marc Viggiano, Edward Valovage, and Marcello Distasio. 2004. Secure automatic dependant surveillance. Google Patents.
- [42] Mark Walport. 2016. *Distributed Ledger Technology: Beyond Block Chain*. Government Office for Science.
- [43] Yanhan Wu, Xin Lu, and Zhijun Wu. 2021. Blockchain-based trust model for air traffic management network. In *2021 IEEE 6th International Conference on Computer and Communication Systems (ICCCS)*. IEEE, 92–98.
- [44] Zhijun Wu and Jiawei Liu. 2022. Blockchain-based trusted avionics authentication for secure ground-to-air communication. In *2022 IEEE/AIAA 41st Digital Avionics Systems Conference (DASC)*. IEEE, 1–9.
- [45] Oliver Wyman. [n. d.]. Global Fleet & MRO Market Forecast 2022-2032. Retrieved from <https://www.oliverwyman.com/our-expertise/insights/2022/feb/global-fleet-and-mro-market-forecast-2022-2032.html>. Accessed: 2023-05-30.
- [46] Oliver Wyman. 2022. Aviation Is Poised For A Decade Of Growth, But There Are Headwinds Besides Covid-19. Retrieved from <https://www.forbes.com/sites/oliverwyman/2022/02/24/why-aviation-will-grow-for-a-decade-but-there-are-headwinds-besides-covid-19/>. Accessed: 2023-05-30.
- [47] Anatoly Yakovenko. 2018. Solana: A new architecture for a high performance blockchain v0. 8.13. *Whitepaper* (2018).
- [48] Anatoly Yakovenko. 2019. 8 Innovations that Make Solana the First Web-Scale Blockchain. Retrieved from <https://medium.com/solana-labs/7-innovations-that-make-solana-the-first-web-scale-blockchain-ddc50b1defda>. Accessed: 11-05-23.
- [49] Zibin Zheng, Shaoan Xie, Hongning Dai, Xiangping Chen, and Huaimin Wang. 2017. An overview of blockchain technology: Architecture, consensus, and future trends. In *2017 IEEE International Congress on Big Data (BigData Congress)*. 557–564. DOI: <https://doi.org/10.1109/BigDataCongress.2017.85>

Received 19 September 2024; revised 3 July 2025; accepted 29 July 2025